ARCHITECTURE GUIDE · VERSION 0.1.0

Cereal, from the inside

How the app is built: its processes, data, audio and transcription pipeline, AI layer and agent. Then how Granola, the app it is modelled on, builds the same things and the parts Cereal doesn't have yet.



Written from the source in `~/Desktop/projects/cereal` and a static reverse-engineering pass (REA) over Granola 7.663.2 and 7.668.0.

20 October 2026

Contents

PART I · CEREAL

1 What Cereal does 5
2 Processes and the shape of the app 7
3 The IPC contract 10
4 Storage: one SQLite file 13
5 Settings, keys and secrets 15
6 Audio capture and meeting detection 17
7 Transcription 19
8 Enhance notes 23
9 The LLM layer 27
10 The agent, part 1: the core 29
11 The agent, part 2: tools, approvals and the chat 34
12 The agent, part 3: the Assistant 39
13 The interface 42
14 Widget, calendar and attachments 45
15 Building, testing and shipping 47
16 One meeting, end to end 50

PART II · GRANOLA

17 How Granola was studied, and its overall shape 52
18 Identity: sign-in, tokens and deep links 54
19 The sync engine 56
20 Sharing and shareable links 58
21 Google connections 60
22 Integrations, workspaces and updates 62
23 The copied features, Granola's way 64

PART III · THE GAP

24 Capability by capability 66
25 A roadmap for the missing pieces 68
A Source file map 70
B Glossary 72
C Evidence and further reading 73

How to read this guide

The guide has three parts.

- **Part I, Cereal**, goes through the app layer by layer, in the order data flows: processes and IPC, storage, audio, transcription, notes generation, the LLM layer, the agent, the interface, and packaging. It ends by following one meeting from start to finish, end to end.
- **Part II, Granola**, explains how Granola does the things Cereal doesn't: accounts and sign-in, server sync, sharing and shareable links, Google connections, integrations, workspaces and updates. It also recaps how Granola runs the features Cereal copied.
- **Part III, the gap**, compares the two capability by capability and sketches how Cereal could add the missing pieces without giving up being local-first.

You can read the chapters on their own. The agent chapters (10–12) are the densest; read chapters 2, 3 and 9 first.

Conventions

Mark	Meaning
`src/main/recorder.ts`	A path in Cereal's repository, relative to its root.
O observed	A Granola fact seen in the shipped bytes: REA output, or the extracted bundle read as inert text. Nothing from Granola was run.
I inferred	A conclusion drawn from observed facts, not seen directly.
? unknown	An open question the static analysis could not settle.

KEY IDEA BOXES

The one thing to remember from a section.

IN THE CODE

Where to look in the source, and what to read first.

GRANOLA

How Granola does the same thing, when it differs in an instructive way.

NOT IN CEREAL

Something Granola has that Cereal lacks.

Clean-room note

Cereal reimplements Granola's *behaviour*. The Granola material in this guide records names, data shapes, endpoint names, enums and timings. It contains no Granola source code, prompt text, copy, icons or images, and Cereal ships none. Granola's prompts live on its servers and were never seen. The evidence files are in `~/re/granola/evidence/`; never commit them.

PART I

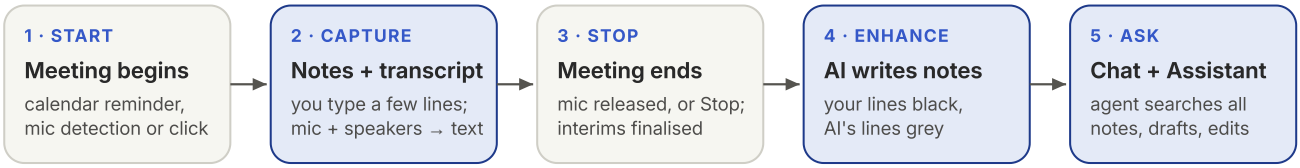
Cereal

A Linux AI meeting notepad in about 22,000 lines of TypeScript: an Electron app with a SQLite database, a PipeWire audio pipeline, two transcription engines, four LLM providers and an agent that runs in its own process.

CHAPTER 1

What Cereal does

Before the architecture, the product loop it serves. Every subsystem in this guide exists to make one of these steps work.



Steps 2 and 4 are where Cereal earns its keep. Step 2 is a real-time pipeline (chapters 6–7); step 4 is one long LLM call with careful prompt construction and attribution (chapter 8). Step 5 is the agent (chapters 10–12).

Figure 1.1 The product loop. Navy marks the steps that call an AI model.

Cereal is a clean-room Linux reimplementation of **Granola**, an AI notepad for meetings. The idea: you take sparse notes during a call, the app transcribes both sides of the conversation without a bot joining, and afterwards an LLM merges your notes with the transcript into clean, structured notes. Your own words stay in full colour; what the AI added is grey.

Around that loop sit the features that make it usable every day:

- **Two transcription engines.** Deepgram streaming in the cloud, or whisper.cpp locally (chapter 7).
- **Lecture mode.** Microphone only, labelled "Speaker", with study-note templates.
- **Templates.** Auto, 1:1, Stand-up, Customer call, Interview, Lecture, plus your own. They decide the headings of the enhanced notes.
- **Transcript Summary.** Hover any enhanced line to ask "where did this come from?" and get quotes that jump into the transcript.
- **Ask Cereal.** An agentic chat, per note or across all notes, with citations, recipes (`/follow-up-email`), @-mentions, file attachments, dictation and a model picker.
- **The Assistant.** A personal agent that hands work to parallel tasks, runs routines on a schedule, keeps a memory and loads skills.
- **Google Calendar.** Upcoming meetings, reminders, and attendee names that improve transcription and prompts.
- **Meeting detection and a floating widget**, so recording keeps going while you're in Zoom.

KEY IDEA

Cereal is **local-first and single-user**. There is no Cereal server. Your notes, transcripts and agent state live in one SQLite file. The only network traffic goes to services you configured with your own keys: Deepgram, an LLM provider, and Google. Granola is the opposite: a thin client over many backend services (Part II). Most of what Cereal "lacks" (sharing, sync, sign-in, integrations) needs a server, and that is the real design question of Part III.

The stack in one table

Concern	Choice	Why
Shell	Electron 44 (Chromium + Node), built with electron-vite 6 and Vite 8	Same model as Granola; one codebase for UI and system work.

Concern	Choice	Why
UI	React 19, Tailwind 4, Base UI primitives, `motion`, `sonner`, TipTap 3 (ProseMirror) editors, zustand stores	Matches Granola's "oats" design system and libraries.
Storage	SQLite through better-sqlite3 13 (synchronous, WAL, FTS5)	Fast, simple, one file; sync calls are fine in main for one user.
Audio	PipeWire's `pw-record` and `pw-dump` as child processes	Captures speakers through the sink monitor; no native module needed.
Speech-to-text	Deepgram nova-3 over WebSocket, or `whisper-server` (whisper.cpp) on localhost	Cloud quality or full privacy.
LLMs	Anthropic (default `claude-opus-5-5`, fast `claude-haiku-5-5`), OpenAI Responses API, Gemini, Ollama	Provider-neutral layer with tool calling and files.
Validation	zod 4 (tool inputs, converted to JSON Schema for the model)	One schema for runtime checks and model tool specs.
Distribution	Flatpak `dev.cereal.app` on the Freedesktop 26.08 runtime and the Electron BaseApp	Sandboxed; bundles whisper.cpp.
Tests	vitest run under Electron's Node; a Playwright-driven end-to-end smoke test with fake servers	better-sqlite3 is built for Electron's ABI.

The source tree

```
src/
  main/            Electron main process: everything with system or network access
    index.ts       tiny dispatcher: app.ts, or widget/host.ts when started with --cereal-widget-host
    app.ts         composition root: builds every service, registers IPC, owns windows/tray/lifecycle
    ipc.ts  windows.ts  settings.ts  settings.pure.ts  recorder.ts  detector.ts  calendar.ts
    audio/         pipewire.ts: capture, device list, meeting-app detection
    transcription/ deepgram.ts, whisper.ts, segmenter.ts, models.ts, dictation.ts, types.ts
    ai/            service.ts (enhance), variables.ts, prompts.ts, transcriptSummary.ts
    llm/           provider.ts (interfaces), index.ts (factory), anthropic/openai/gemini/ollama.ts,
files.ts
    attachments/   service.ts, extract.ts (PDF, docx, xlsx text)
    agent/         runtime.ts, backend.ts, protocol.ts, chat.ts, assistant.ts, schedule.ts, files.ts,
                   prompts.ts, tool.ts, host.ts, worker-host.ts, worker/entry.ts, tools/*
    db/            index.ts (open + migrate), migrations.ts, store.ts, agent-store.ts
    widget/        controller.ts, host.ts, protocol.ts, placement.ts
  preload/         index.ts: contextBridge → window.cereal
  renderer/        index.html + widget.html; src/ (App, views, components, oats primitives, lib)
  shared/          imported by all three: types, IPC contract, agent types + reducer, transcript,
markdown…
tests/             13 vitest suites (~150 cases) + e2e/smoke.mjs
flatpak/  scripts/flatpak.sh  docs/granola-*.md
```

CHAPTER 2

Processes and the shape of the app

Electron gives you processes with very different powers. The first architectural decision is what runs where.

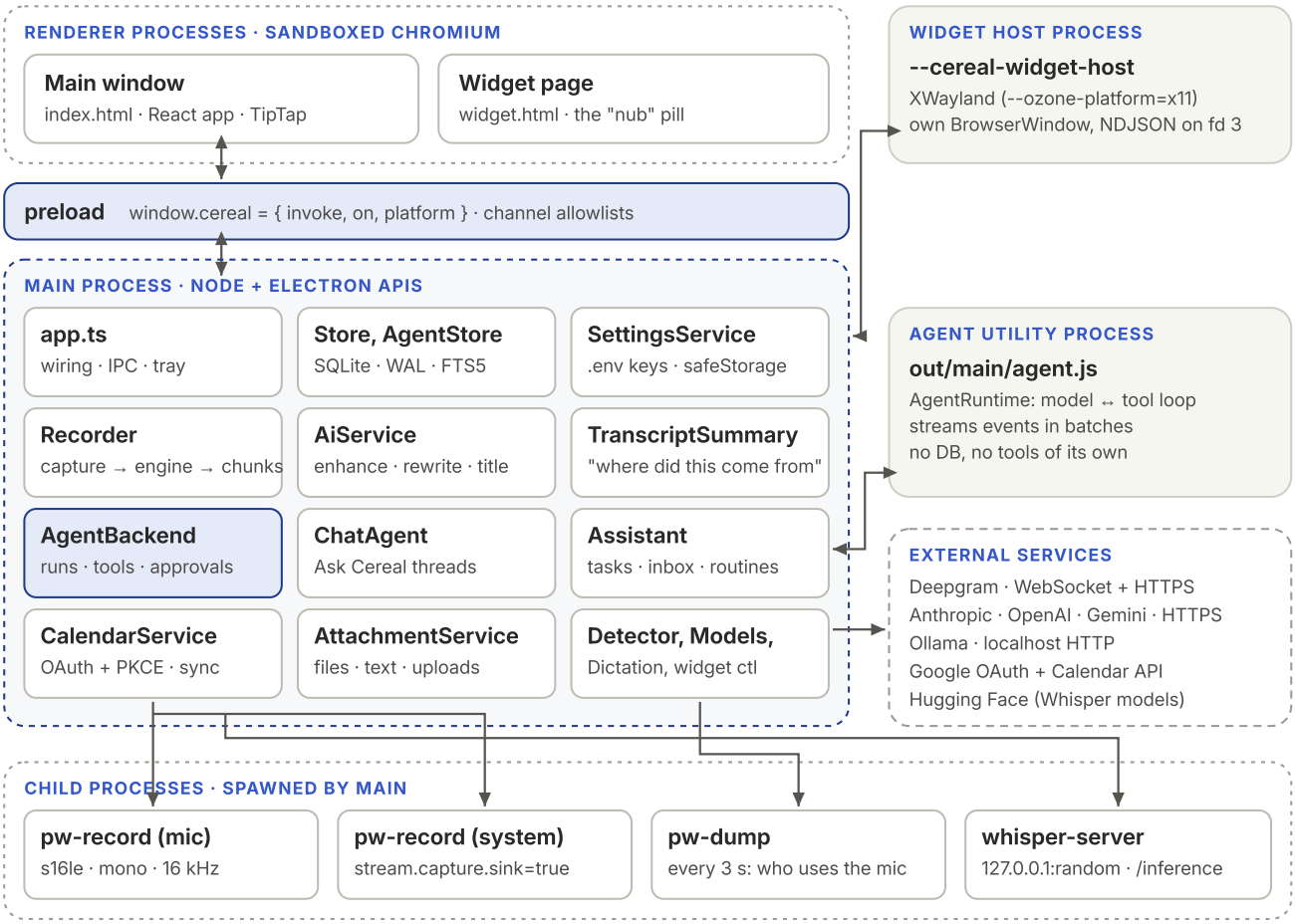


Figure 2.1 Cereal's processes. Only the main process can touch the disk, the network or devices. Renderers reach it through the preload bridge; the agent runtime reaches it through an RPC over the utility process's message port.

Why this split

Electron's **main process** is a Node.js program with access to everything. A **renderer** is a Chromium tab: it draws the UI but, configured as Cereal configures it, cannot touch the filesystem or Node at all. Cereal follows the Electron security baseline:

- **Renderer settings.** Every window has `sandbox: true`, `contextIsolation: true` and `nodeIntegration: false` (`app.ts#createMainWindow`, `windows.ts#widgetWindowOptions`).
- **The bridge.** The only door is the preload script. It exposes `window.cereal` with three members and refuses any channel name not in a fixed allowlist.
- **Sender check.** Main double-checks the sender. `ipc.ts#handle` throws *Untrusted IPC sender* unless the calling frame's URL is one of Cereal's own pages (`file://`, or the Vite dev server in development).
- **Navigation.** `windows.ts#hardenWindow` denies `window.open` and in-app navigation, sending `http(s)` links to the system browser instead.
- **Permissions.** Microphone, notifications and clipboard are granted only to Cereal's own pages, through `session.setPermissionRequestHandler`.

- **CSP.** Both HTML pages carry a strict Content-Security-Policy: scripts from `'self'` only, images also from `data:`, `blob:`, `https:` and the attachment scheme.

Heavy or risky work goes into **other processes**, so a crash or a long computation can't freeze the UI:

- **The agent loop** runs in an Electron `utilityProcess` (`out/main/agent.js`). Its model calls stream for minutes, and a crash there just fails the run (chapter 10).
- **Audio capture and local speech recognition** are separate executables (`pw-record`, `whisper-server`) that main spawns and talks to over pipes and localhost HTTP.
- **The floating widget** on Wayland runs in a second instance of the Electron binary under XWayland, because Wayland compositors won't let a window pin itself to the screen edge and stay on top (chapter 14).

GRANOLA

Granola splits even more aggressively: about a dozen utility processes (`audio_process`, `sqlite_process`, `local_asr_process`, `local_diarization_process`, `mic_monitor_v2_process`, `accessibility_process` and others), each in its own folder under `dist-electron/`. Its SQLite runs in a utility process with a 4-thread pool and an encrypted database. Cereal keeps SQLite in main because a single user's dataset is small and better-sqlite3 calls take microseconds. That keeps every query synchronous and simple, at the cost of main briefly blocking on a big write.

One binary, four entry points

`electron.vite.config.ts` builds the main side with four Rollup inputs, plus a preload and two renderer pages:

Output	Source	Role
`out/main/index.js`	`src/main/index.ts`	`package.json#main`. Dispatches on argv: `--cereal-widget-host` → `import('./widget/host')`, otherwise `import('./app')`.
`out/main/app.js`	`src/main/app.ts`	The application.
`out/main/widget-host.js`	`src/main/widget/host.ts`	The XWayland widget process.
`out/main/agent.js`	`src/main/agent/worker/entry.ts`	The agent utility process.
`out/preload/index.js`	`src/preload/index.ts`	The bridge, shared by both pages.
`out/renderer/{index,widget}.html`	`src/renderer/*.html`	The React app and the widget page.

Shared code lands in `out/main/chunks`. Because entries sit at different depths, helpers that resolve files take the *caller's* `__dirname` as `baseDir` (`windows.ts#loadPage`). `externalizeDepsPlugin` leaves `node_modules` as real `require`s, which matters for better-sqlite3's native `.node` file. The alias `@shared` points at `src/shared` in all three builds, so types and pure logic are written once.

app.ts: the composition root

`app.ts` is about 660 lines and reads top to bottom like a wiring diagram. It is the one file that knows every service; every other module receives what it needs through its constructor. A typical service looks like this:

```
const recorder = new Recorder({
  store,
  settings: () => settings.get(),
  createEngine,                                  // Deepgram or Whisper, decided per recording
  createCapture: (opts) => new PipeWireCapture(opts),
  emitChunks: (documentId, chunks) => broadcast('transcript:chunks', { documentId, chunks }),
  emitState: (state) => { broadcast('recording:state', state); widget.setRecording(state);
updateWidget(); updateTray() },
  emitLevels: (levels) => { broadcast('audio:levels', levels); widget.setLevels(levels) },
  sampleRate: SAMPLE_RATE
})
```

KEY IDEA: DEPENDENCY INJECTION BY CALLBACKS

Services never import Electron's `BrowserWindow` or call `broadcast` themselves. They get `emit*` callbacks and factory functions. That is why `tests/services.test.ts` can run the real `Recorder` with a fake capture and a fake engine, and `tests/agent.test.ts` can run the real agent backend with a scripted LLM, all under plain Node without windows. When you add a service, follow the same pattern: pure logic in `src/shared` or a class with injected deps, wired only in `app.ts`.

Startup and shutdown

1. **Module load** (before `ready`).
   - Set the app name and take the single-instance lock; a second launch quits and focuses the first one through `second-instance`.
   - In Flatpak, enable Chromium's `SecretPortalKeyProviderUseForEncryption`.
   - Register the privileged `cereal-attachment://` scheme.
   - Set `userData` (or `CEREAL_USER_DATA`).
   - Open the database, which runs migrations, then construct every service.
2. **`app.whenReady()`**.
   - Start decrypting tokens. Only the calendar waits for them, because the keyring can be slow.
   - Serve the attachment protocol and install permission handlers.
   - Register about 90 IPC handlers, then create the main window and the tray.
   - Start meeting detection.
   - `agent.recover()` fails runs that were mid-flight when the app quit, but keeps runs waiting for approval resumable.
   - Prune agent events older than 14 days.
   - Start three timers: the Assistant `tick` every 30 s (routines and batched inbox), meeting reminders every 30 s, and calendar sync every 5 min.
3. **While recording.** Closing the window only *hides* it, so the meeting keeps transcribing and the floating widget appears. `window-all-closed` quits only when idle.
4. **Quit.** If a recording is live, `will-quit` is cancelled once, the recorder stops (flushing final transcript chunks), and then the app quits for real. After that, detector, dictation, the agent process and the widget host are stopped and the database is closed.

CHAPTER 3

The IPC contract

Main and the renderers share no memory. Everything the UI does goes through 82 request channels and 21 push channels, defined once and typed end to end.

Two kinds of channel

`src/shared/ipc.ts` is the single source of truth. It defines two TypeScript interfaces:

- **`InvokeMap`: request/response.** Each key is a channel name in Granola's `domain:action` style (`documents:update`, `chat:send`, `asr:download`). Each value is a function type: its parameters are what the renderer sends and its return type is what comes back. The renderer calls them with `ipcRenderer.invoke`; main answers with `ipcMain.handle`.
- **`EventMap`: push.** Main tells every window that something changed: `transcript:chunks`, `recording:state`, `panel:updated`, `chat:message`, `agent:event`, `documents:changed`.

```
export interface InvokeMap {
  'documents:update': (id: string, patch: DocumentPatch) => CerealDocument
  'panels:enhance': (documentId: string, templateSlug?: string | null) => Panel
  'chat:send': (input: ChatSendRequest) => { threadId: string; messageId: string }
  'agent:events': (runId: string, afterSequence: number) => AgentEventRecord[]
  // … 78 more
}
export interface EventMap {
  'transcript:chunks': { documentId: string; chunks: TranscriptChunk[] }
  'agent:event': AgentEventRecord
  // … 19 more
}
export const INVOKE_CHANNELS = [ /* every key, as a runtime array */ ] as const satisfies readonly
InvokeChannel[]
```

The `satisfies` check catches misspelled names in the runtime list at compile time. It can't catch a key that is missing from the list: the preload would then reject a channel that typechecks, and the error appears the first time the renderer calls it.

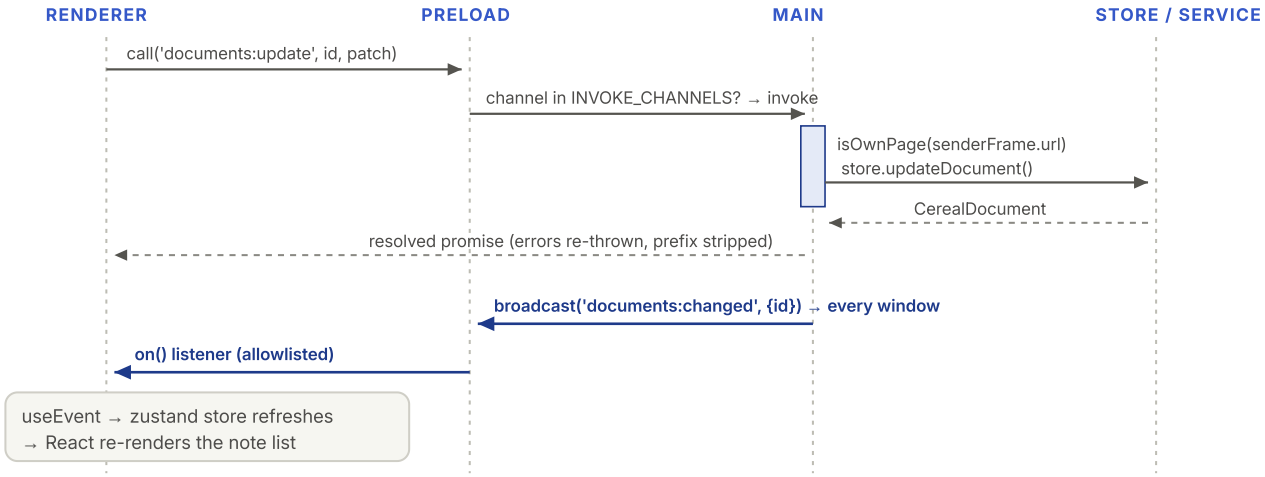


Figure 3.1 One round trip and one push. Requests are point-to-point; pushes go to all windows (the main window and an in-process widget), and each decides whether it cares.

The three pieces of plumbing

Preload: `src/preload/index.ts` (21 lines)

It builds two `Set`s from `INVOKE_CHANNELS` and `EVENT_CHANNELS` and exposes `window.cereal` through `contextBridge.exposeInMainWorld`:

- **`invoke(channel, …args)`**: rejects unknown channels, otherwise `ipcRenderer.invoke`.
- **`on(channel, listener)`**: wraps the listener so the renderer never sees the raw `IpcRendererEvent`, which would leak `sender`, and returns an unsubscribe function.
- **`platform`**.

Its type is `CerealBridge`, whose generic signatures make `window.cereal.invoke('panels:enhance', id)` return `Promise<Panel>` with no casts.

Main: `src/main/ipc.ts` (24 lines)

- **`handle(channel, handler)`** registers an `ipcMain.handle` whose handler must match `InvokeMap[channel]`. It may return the value or a promise of it. Every call first runs the sender validator.
- **`broadcast(channel, payload)`** sends to every live `BrowserWindow`.

The widget's XWayland host process reuses the same `handle` for the few widget channels it serves.

Renderer: `src/renderer/src/lib/api.ts`

- **`call()`** wraps `invoke` and strips Electron's *"Error invoking remote method 'x': Error:"* prefix, so a thrown `new Error('Note not found')` in main reaches a toast as exactly that text. This is how user-facing errors travel. Main throws friendly messages, for example *"Set DEEPGRAM_API_KEY in …/.env"*, and the UI shows them as they are.
- **`useEvent(channel, listener)`** is a React hook that subscribes for a component's lifetime. It keeps the latest listener in a ref, so handlers can close over fresh state without re-subscribing.

The channel map by domain

Domain	Invoke channels	Pushes
Notes	`documents:list|get|create|update|delete|for-event`, `folders:*` (6), `templates:*`, `recipes:*`, `search:query`	`documents:changed`, `folders:changed`
Recording	`recording:start|stop|state`, `transcript:get|clear`, `audio:devices`, `asr:models|download|delete|test-deepgram`, `dictation:transcribe`	`recording:state`, `audio:levels`, `transcript:chunks`, `asr:models-changed`, `meeting:detected`
Enhanced notes	`panels:list|enhance|rewrite|delete|update`, `transcript-summary:start|cancel`	`panel:updated`, `transcript-summary:update`
Chat	`chat:threads|messages|send|retry|stop|delete-thread|tool-applied|attach|remove-attachment|models|mention-search`	`chat:message`, `chat:thread`, `chat:attachment`
Agent and Assistant	`agent:decide|events|sources`, `assistant:open|thread|send|stop|task-stop|routines|routine-update|routine-delete|skills|files|file-read|open-folder|reset`	`agent:event`, `assistant:message|thread|home|routines-changed`
Settings	`settings:get|update`, `secrets:status|open-env`, `llm:test|models`, `env:status`	`settings:changed`

Domain	Invoke channels	Pushes
Calendar	`calendar:connect|disconnect|upcoming|sync`	`calendar:changed`
Shell	`app:open-external|focus-main|quit`, `window:chrome`, `widget:resize|action|state`	`navigate`, `widget:state`

IN THE CODE: ADDING A FEATURE THAT CROSSES THE BOUNDARY

1. Add the signature to `InvokeMap` (or `EventMap`) in `shared/ipc.ts`.
2. Add the name to `INVOKE_CHANNELS` (or `EVENT_CHANNELS`). Forget this and the preload refuses the call at runtime.
3. `handle('x:y', …)` inside `registerIpc()` in `app.ts`, delegating to a service.
4. In the renderer, `await call('x:y', …)`, or `useEvent('x:changed', …)` for pushes.

Push, then pull

Many pushes carry only an identifier. `documents:changed {id}` makes the store re-fetch the list with `documents:list`, and the open note re-fetches itself. Others carry the full object, because it changes many times a second:

- `panel:updated` streams an enhanced panel as it generates.
- `chat:message` carries a message with its outputs.
- `transcript:chunks` carries only the chunks that changed.

The rule of thumb: if it changes many times a second, push the data; otherwise push a hint and let the view query. That keeps a single source of truth (SQLite) and makes windows that open later trivially consistent.

GRANOLA

Granola's preload exposes about 365 members across two `contextBridge` APIs `O`, most of them generated from dynamic channel names. It also mirrors a large application *cache* between processes: main owns a Redux-style state (`cache-v6.json`, version 9), and renderers send actions over `cache:dispatch` and receive `cache:committed` `O`. Cereal's narrower contract (typed request/response plus hints) is simpler, because SQLite in main is already the shared state.

CHAPTER 4

Storage: one SQLite file

Everything Cereal knows lives in `cereal.db` in the profile folder: notes, transcripts, enhanced notes, chats, the agent's runs and events, settings. Two classes front it.

Opening the database

`db/index.ts#openDatabase` opens the file with better-sqlite3 and sets three pragmas:

- `journal_mode = WAL`: readers don't block the writer, and commits are cheap.
- `foreign_keys = ON`: `ON DELETE CASCADE` actually cascades.
- `busy_timeout = 5000`.

Then it migrates. `migrations.ts` exports an ordered array, and the database's `PRAGMA user_version` records how many have run. Each pending migration runs inside a transaction together with the version bump, so a failed migration leaves the database untouched. A migration is usually an SQL string. Migration 5 is a function, because it had to transform data: it converts `generated_lines` from line indices to line texts (chapter 8).

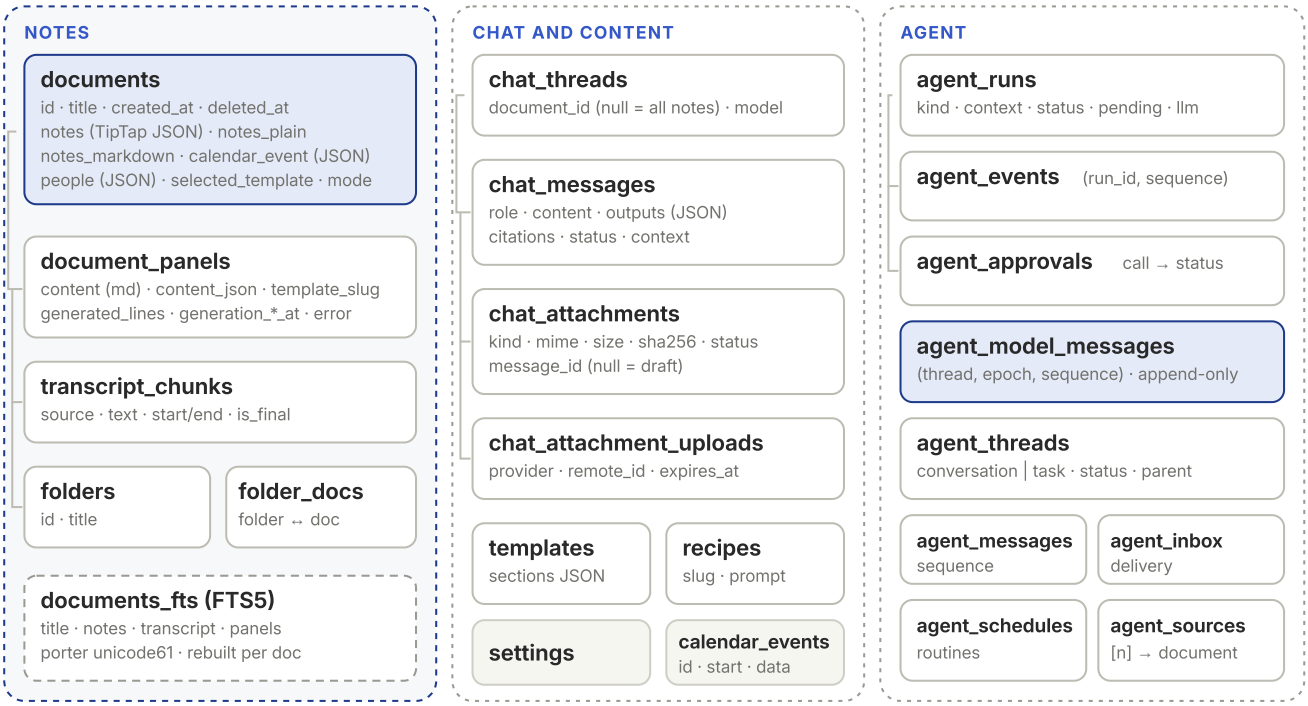


Figure 4.1 The schema after migration 5: 23 tables. Lines on the left of a group show foreign keys that cascade on delete. The agent tables serve both the chat (runs of kind `chat`, keyed by `chat_threads.id`) and the Assistant (runs of kind `assistant`, keyed by `agent_threads.id`).

Design choices worth noticing

- **Granola's column names.** `documents` keeps `notes`, `notes_plain`, `notes_markdown`, `selected_template` and `deleted_at`. Transcript chunks keep Granola's chunk shape, with ISO timestamps and `source: microphone|system`. The shapes in `shared/types.ts` mirror them, so prompts and UI code read like Granola's.
- **The editor's JSON is stored as is.** `notes` is the TipTap/ProseMirror document. Two derived copies, plain text and Markdown, are saved alongside on every edit. Search, prompts and the agent use the derived copies and never parse ProseMirror in SQL.

- **JSON columns for nested data.** `calendar_event`, `people`, `outputs`, `citations` and `sections` are JSON text, parsed by a tolerant `parse()` helper that falls back on bad JSON. An expression index on `json_extract(calendar_event, '$.id')` makes "find the note for this event" fast.
- **Soft delete for notes** (`deleted_at`), hard delete for everything else. Every list query filters `deleted_at IS NULL`, and deleting removes the note's FTS row immediately.
- **Settings are one JSON blob** under the key `settings`, merged over `DEFAULT_SETTINGS` on load (`deepMerge`), so a new setting gets its default after an upgrade without a migration.

Full-text search

`documents_fts` is an FTS5 table with four indexed columns (title, notes, transcript and enhanced panels) and the `porter unicode61` tokenizer, so "meetings" finds "meeting". It isn't maintained by triggers. `Store#reindex(id)` deletes and re-inserts the note's row whenever the title, notes, transcript (on finalise) or a finished panel changes. A query goes through `toFtsQuery`, which lowercases the text, splits on non-letters, drops one-letter words, keeps 12 terms and ORs them as quoted prefix terms (`"budget"* OR "pilot"*`). That makes any user input safe FTS syntax. Results are ranked with

```
ORDER BY bm25(documents_fts, 0, 10.0, 4.0, 1.0, 2.0)   -- document_id, title, notes, transcript, panels
```

so a hit in the title outweighs one in the user's notes, which outweighs the enhanced notes, which outweighs the raw transcript. The palette (`Ctrl K`) and the agent's `searchMeetings` tool both use it, with `snippet()` marking matches with «».

GRANOLA

Granola's local database has the same core tables plus `ydocs`, `document_attendees`, `document_tags`, `pre_meeting_briefs`, `speaker_name_suggestions` and `dictation_history`. It is defined with Drizzle migrations and queried with Kysely `o` . Its FTS uses the `trigram` tokenizer over title and notes only, and its `documents` table has about 50 columns, many of them for sharing and server sync: `visibility`, `sharing_link_visibility`, `has_shareable_link`, `is_shared_direct`, `document_user_role`, `workspace_id`, `ydoc_state` `o` . Chapters 19 and 20 cover those. The database file itself is encrypted (SQLCipher through `better-sqlite3-multiple-ciphers`) `o` ; Cereal's is not.

Two store classes

- **`Store`** (`db/store.ts`, 660 lines) covers documents, transcript, panels, folders, templates, recipes, chat threads, messages and attachments, calendar, settings and search. Its constructor *seeds the built-ins*: an upsert that refreshes built-in templates and recipes on every launch, but never overwrites user-made ones (`WHERE templates.builtin = 1`).
- **`AgentStore`** (`db/agent-store.ts`) covers model transcripts, runs, events, approvals, Assistant threads and messages, inbox, routines and citation sources. Chapter 10 explains each table.

Both are synchronous and use prepared statements, with `db.transaction(…)()` wherever several rows must change together, for example `upsertChunks` or `appendTranscript`.

CHAPTER 5

Settings, keys and secrets

Three kinds of configuration with three storage strategies: preferences in SQLite, API keys in a `.env` file the user edits, and the one token Cereal obtains itself, encrypted by the desktop keyring.

What	Where	Why
Preferences	`settings` table, key `settings`	Plain data. `SettingsService.get()` serves a cached copy; `update(patch)` deep-merges, saves and broadcasts `settings:changed`.
API keys: Deepgram, Anthropic, OpenAI, Gemini, Google client secret	`<userData>/.env`, mode 0600	The user supplies them and Cereal never writes them. Re-read on every lookup, so edits apply without a restart.
Google refresh token	`settings` table, key `secret:googleRefreshToken`, as `enc2:…`	Obtained by the app at sign-in. Encrypted with Electron's async `safeStorage`: the OS keyring, or in Flatpak the Secret portal.

The .env file

- **Creation.** `ensureEnvFile()` writes a commented template on first launch. On later launches it appends any section whose variable a newer version reads but the file doesn't mention yet (`missingEnvSections`), so upgrades never clobber the user's edits.
- **Lookup.** `getSecret(name)` reads the file each time with `readEnvText`, which is defensive:
  - It opens with `O_NONBLOCK` and refuses anything that isn't a regular file, so a FIFO can't hang the main process.
  - If the mode has any group or other bits, it `fchmod`s the file back to 0600. Editors that save by rewriting the file often widen permissions.
- **Precedence.** The `.env` value wins over the process environment. `GOOGLE_API_KEY` is accepted as an alias for `GEMINI_API_KEY`.
- **Status.** Settings shows which keys are set (`secrets:status`, booleans only) and has an **Open .env** button. Keys never cross into the renderer.

The token cipher

`settings.pure.ts` holds the pure, testable logic. It works against a `TokenCipher` interface, which is Electron's async `safeStorage` in the app and a fake in tests:

- **`sealToken`** produces `enc2:<base64>` when encryption is really available, otherwise `plain:<base64>`. Settings then warns that the sign-in is stored unencrypted.
- **The "really available" check.** Chromium can claim it encrypts while using a fixed fallback key: its `v10` prefix. `cipher.available()` encrypts a probe and treats a `v10` result as no protection at all.
- **`openToken`** reads three formats: `enc2:` (async API), the legacy `enc:` (sync API) and `plain:`. It reports whether to *reseal*: the format is old, the keyring rotated its key, or a keyring appeared since a plain value was stored. `loadTokens()` then rewrites the value in the best format available.

WHY ASYNC SAFESTORAGE

Inside Flatpak, Electron's synchronous `safeStorage` gets a new random key on every start, so a token encrypted yesterday can't be decrypted today. The async API, together with the Chromium feature `SecretPortalKeyProviderUseForEncryption` (enabled in `app.ts` only when `FLATPAK_ID` is set), gets a stable per-app key from the desktop's Secret portal. The app gets no access to the rest of the keyring.

GRANOLA

Granola uses *envelope encryption* `O` :

- **The key.** A random 32-byte data-encryption key (DEK), wrapped with async `safeStorage`, or on packaged macOS a native `keychain.node`, and saved as `storage.dek`.
- **The data.** Whole JSON stores (auth tokens and the app cache) are streamed through AES-256-GCM with that key: a 12-byte IV, then ciphertext, then a 16-byte tag. They are written as `<name>.enc` through a temp file and a rename.
- **Recovery.** An unreadable or wrong-length DEK is deleted and a new one generated, which means signing in again.

Chapter 18 shows what is inside those stores.

CHAPTER 6

Audio capture and meeting detection

Cereal hears both sides of a call without joining it. The trick is PipeWire's sink monitor, and the same PipeWire graph also tells Cereal when a call starts.

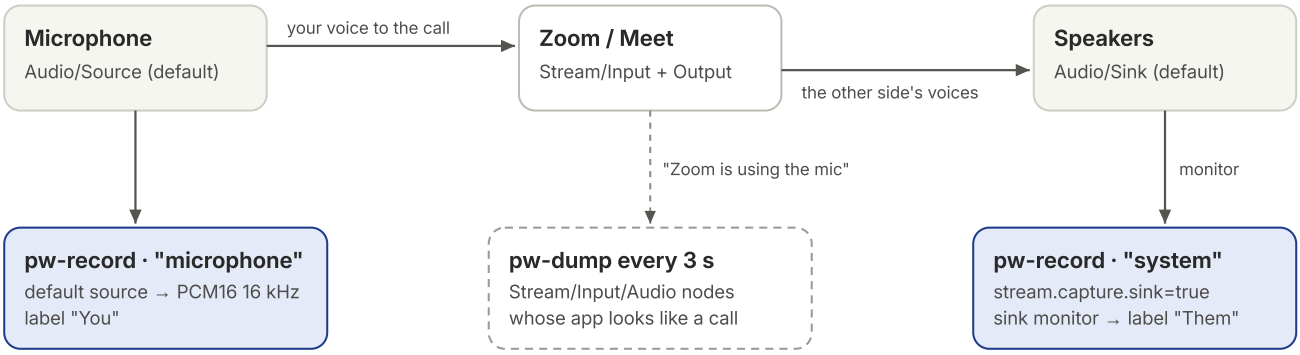


Figure 6.1 The PipeWire graph during a call. Cereal adds two recording streams and reads the graph to detect meetings. Nothing is injected into the call.

PipeWireCapture

`audio/pipewire.ts#PipeWireCapture` is an `EventEmitter` around one `pw-record` child process:

```
pw-record --raw --format s16 --rate 16000 --channels 1 --latency 50ms \
  -P '{ application.name="cereal-capture" node.description="Cereal system" stream.capture.sink=true }' \
  [--target <node.name>] -
```

- **Format.** Raw signed 16-bit little-endian, mono, 16 kHz, on stdout. That is what both Deepgram (`linear16`) and whisper.cpp want, so no resampling happens in JavaScript.
- **The system stream** sets `stream.capture.sink=true`. PipeWire then records the default *output* device's monitor, which is everything you hear. This is the Linux counterpart of Granola's Windows "render loopback".
- **Our own tag.** `application.name="cereal-capture"` labels Cereal's streams so meeting detection can ignore them.
- **Sample alignment.** A pipe read can end mid-sample. `onData` carries an odd trailing byte over to the next read.
- **Levels.** It accumulates the sum of squares and emits `level`, an RMS from 0 to 1, every 1,600 samples (100 ms). The Recorder sends both levels to the UI every 100 ms as `audio:levels`, which drive the "dancing bars". The renderer applies a noise gate first (`shared/barLevels.ts`), because a raw PipeWire microphone hisses at RMS 0.015–0.025 and the bars would otherwise dance in silence.
- **Resilience.** If `pw-record` exits while it should be running (a device unplugged, PipeWire restarted), it respawns up to five times with a growing delay of 0.5 s × n. The counter resets as soon as data flows again. A failing *system* stream only adds a warning ("System audio unavailable"); a failing *microphone* stops the recording.

NO ECHO CANCELLATION

Without headphones, the speakers' audio also reaches the microphone and is transcribed twice, once as "You" and once as "Them". Granola cancels echo natively; Cereal tells you to use headphones. PipeWire's `echo-cancel` module is a possible future improvement.

Meeting detection

- **Finding calls.** `activeMeetingApps()` runs `pw-dump` and keeps nodes with `media.class = Stream/Input/Audio`, which are apps *recording* audio. It skips sink-monitor streams and Cereal's own. It matches `application.name` plus `application.process.binary` against a short table: Zoom, Teams, Slack, Discord, Webex and Skype, plus Chromium- and Firefox-family browsers, reported as "Browser call".
- **The state machine.** `detector.ts#MeetingDetector` polls every 3 s. When the set of call apps goes from empty to non-empty, it fires `onStart(app)`. When it has been empty for 20 s straight, it fires `onEnd()`. The grace period absorbs a browser briefly releasing the mic.
- **What `app.ts` does with it.**
  - On start, if nothing is recording, the floating widget switches to "prompt" mode with the current calendar event's title and asks *Take notes?*.
  - On end, an unanswered prompt is withdrawn. A recording in progress is stopped, with a "Meeting ended" notification, unless the note is a lecture.

GRANOLA

Granola has a native, Rust-written `MicActivityTracker` in `mic_monitor_v2_process`. It emits `session_active` and `session_inactive` per process and follows the meeting app's output device `o` . Capture is native too: two PCM16 streams with echo cancellation, delivered through `audio_process` `o` .

CHAPTER 7

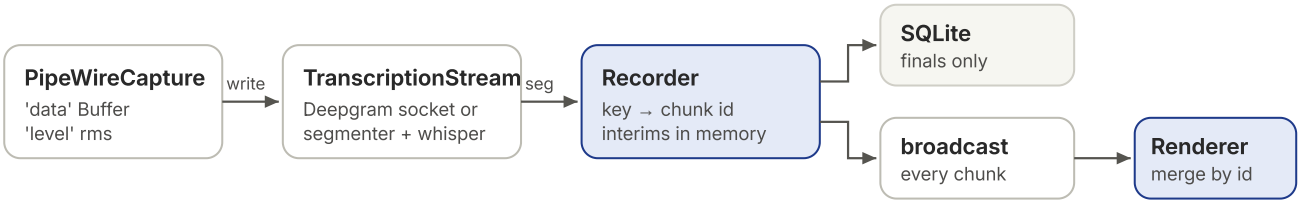
Transcription

Two engines behind one interface, and a Recorder that turns their output into transcript chunks with Granola's exact merge semantics.

The engine interface

```
interface TranscriptionEngine { id: 'deepgram' | 'whisper'; open(opts: StreamOptions):
Promise<TranscriptionStream>; dispose?(): void }
interface TranscriptionStream extends EventEmitter {   // emits 'segment' | 'warning' | 'error'
  write(pcm: Buffer): void; finish(): Promise<void>; close(): void
}
interface EngineSegment { key: string; text: string; startMs: number; endMs: number; isFinal: boolean }
```

The important field is `key`. An engine refines one utterance several times: *"our budget"*, then *"our budget for the"*, then the final *"Our budget for the pilot is $50,000."*. All of these share a key until one arrives with `isFinal`. Times are milliseconds from the start of that source's audio.



One pipe per source: microphone, and system unless lecture mode or system capture is off. Both pipes share one engine (one Deepgram key, one whisper-server process) but have separate streams, so "You" and "Them" are transcribed independently and never mixed.

Chunk = { id, document_id, source, text, start_timestamp, end_timestamp (ISO), is_final }, Granola's shape.
start = pipe.startedAt + seg.startMs: engine time anchored to the wall clock when that source started.

Figure 7.1 The recording pipeline for one source.

The Recorder: `src/main/recorder.ts`

The Recorder is a small state machine: `idle → starting → recording → stopping → idle`. Every change is pushed as `recording:state`, which drives the UI, the tray menu and the floating widget.

start(documentId)

1. **Busy check.** If already recording this note, return. If recording another note, stop that first.
2. **The engine.** Ask `createEngine()` (in `app.ts`) for one, chosen per recording from Settings. It throws a friendly error if the Deepgram key is missing or `whisper-server` can't be found.
3. **Key terms.** Build them from the user dictionary, the user's name and the attendees' names, each under 50 characters. Deepgram biases recognition towards them; Whisper gets them as a prompt.
4. **The pipes.** For each source, open an engine stream, create a capture, wire `data → stream.write`, `segment → onSegment` and the error handlers, then start the capture.
5. **Levels and state.** Start the 100 ms level timer and set the state to `recording`.

onSegment: the interim/final rule

- **Chunk ids.** Each pipe keeps a map from `segment.key` to a chunk id, a random UUID minted the first time the key is seen. Every refinement of the utterance therefore reuses **the same chunk id**.

- **Interim segments** go into an in-memory `interims` map and are broadcast. They are *not* written to SQLite: interims change several times a second.
- **Final segments** remove the key mapping, so the next utterance gets a new id, are upserted into `transcript_chunks`, and are broadcast.

stop()

1. Set the state to `stopping` and stop the captures.
2. Wait for every stream's `finish()`: Deepgram flushes and closes, and Whisper transcribes the last open segment.
3. Mark any interim the engine never finalised as final and save it.
4. `finalizeTranscript` sets `is_final = 1` for the note and reindexes search.
5. Tear down and return to `idle`. The last error survives into the idle state, so the UI can toast it.

Merging on the other side

`shared/transcript.ts#mergeTranscriptChunks` is the reducer both the renderer store and (conceptually) the database use:

- **Upsert by id.** A later interim or the final replaces the earlier one *in place*.
- **Drop empty text.** Chunks with no text are removed.
- **Sort.** By (start, end, id), Granola's ordering.

Live speech therefore grows one line instead of stacking new ones. `loadTranscript` merges the stored finals with any live interims that arrived before the load finished, so opening a note mid-meeting loses nothing.

The same file provides the views of a transcript:

- `groupChunks` makes the chat-bar bubbles: same source, until a chunk ends with a full stop.
- `groupTurns` makes the speaker turns for prompts: same source, at most 15 s gap and 60 s per turn.
- `formatTranscript` turns those into `[mm:ss] You: …` lines.
- `findInChunks` powers Find in transcript.
- `maxNumHeadings` gives the heading budget used in chapter 8.

Deepgram: `transcription/deepgram.ts`

- **Connection.** One WebSocket per source to `wss://api.deepgram.com/v1/listen`, with the header `Authorization: Token <key>`. The query mirrors what Granola sends:
  - `model=nova-3`, `encoding=linear16`, `sample_rate=16000`, `channels=1`.
  - `interim_results`, `smart_format` and `punctuate`; `profanity_filter=false` and `mip_opt_out=true`.
  - `language`. For `multi`, also `endpointing=100`.
  - Up to 100 `keyterm` values (`keywords` for pre-nova-3 models).
- **Buffering and keepalive.** Audio written before the socket opens is queued. A `{"type":"KeepAlive"}` goes out whenever 5 s pass without audio.
- **Utterance keys.** A segment's key is `w<window>`, and the window counter increments after every final result. Each Deepgram "is_final" therefore closes one chunk.
- **Reconnecting with a continuous clock.** Deepgram restarts its timestamps on each new socket. On an unexpected close, the stream adds the audio already sent (bytes ÷ 2 ÷ sample rate) to an `offsetMs`, reconnects up to five times with a growing delay, and warns the UI. Timestamps therefore keep increasing across drops.
- **Finish.** `finish()` sends `{"type":"CloseStream"}` and waits up to 6 s for the server to close, so the last finals arrive. An HTTP 401 or 403 on connect becomes "Deepgram rejected the API key…".

Whisper: local and private

whisper.cpp has no streaming API, so Cereal builds streaming out of three parts.

1. WhisperServer (`whisper.ts`)

- **Startup.** It spawns `whisper-server -m <model> --host 127.0.0.1 --port <free> --threads N --suppress-nst` and polls `GET /` every 250 ms until it answers, for up to 60 s. That loads the model once.
- **Requests.** `transcribe(float32, rate, prompt?)` encodes a WAV and POSTs multipart form data to `/inference` with `temperature=0.0` and `response_format=json`.
- **One at a time.** Requests are chained on a promise, so they run serially and a busy counter is available.
- **Cleaning.** `cleanWhisperText` strips Whisper's non-speech markers such as `[BLANK_AUDIO]` and `(music)`.
- **Finding the binary.** `findWhisperServer` tries the configured path, then `PATH` and a few common folders. In the Flatpak, whisper.cpp v1.9.5 is built into `/app/bin`.

2. The Segmenter (`segmenter.ts`)

The Segmenter is an energy-based voice-activity detector with Granola's timing constants. Audio is cut into 512-sample hops (32 ms at 16 kHz), and a hop counts as speech when its RMS exceeds `max(0.008, noiseFloor × 3)`. The noise floor is an exponential moving average updated only during non-speech.

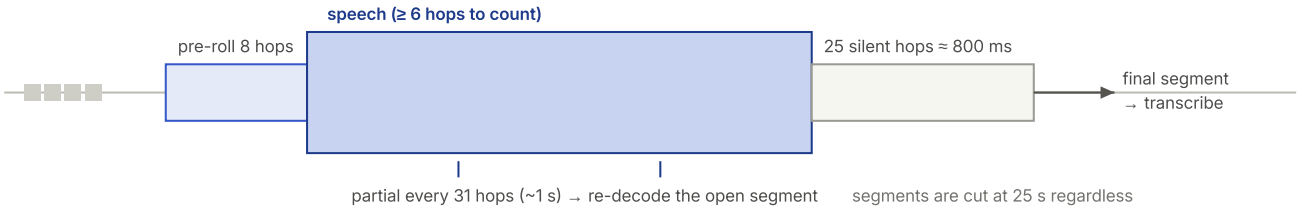


Figure 7.2 How the segmenter turns a stream into segments. The pre-roll keeps the first syllable that triggered detection.

3. WhisperStream and the PrefixStabilizer

- **Partial events.** Each partial is re-transcribed, unless the server is still busy; partials are best-effort, finals always run. The words go through a `PrefixStabilizer`, a "local agreement" policy: only the longest word prefix two successive hypotheses agree on is shown. Text on screen therefore only grows and never flickers between guesses. It is emitted as an interim segment with key `s<segmentId>`.
- **Final events** are transcribed with the key-term prompt (at most 40 terms) and emitted as final. Late partials for a finalised segment are ignored.

Models and dictation

- **Models.** `models.ts` lists six ggml models, from tiny.en (75 MB) to large-v3-turbo-q5_0 (547 MB). They are downloaded on demand from Hugging Face into `<userData>/models`, through a `.part` file renamed when complete, with progress pushed every 250 ms (`asr:models-changed`).
- **Dictation** (Ctrl+T in the chat bar) records in the renderer with the Web Audio API and sends float samples over `dictation:transcribe`. `dictation.ts` uses the same engine choice: Deepgram's *prerecorded* API (one POST of a WAV), or a dedicated WhisperServer kept warm for 2 minutes after the last use so follow-up questions are instant. Audio beyond 10 minutes is cut, matching Granola's auto-send.

GRANOLA

Granola's Deepgram setup is the same in spirit, with three differences [o] :

- **The key** is a short-lived token from Granola's backend (`get-deepgram-token`), cached until it nears expiry.
- **Fallbacks.** There is a fallback dedicated Deepgram host, and AssemblyAI's universal-streaming WebSocket as an alternative provider, with end-of-turn tuning.
- **Local ASR** has two engines in a utility process: NeMo Parakeet through sherpa-onnx, and the same `whisper-server` approach. Voice activity detection uses Silero through onnxruntime.

Cereal's energy VAD is a deliberate simplification.

CHAPTER 8

Enhance notes

Turning rough notes and a transcript into structured notes is one streamed LLM call. The interesting parts are what goes into the prompt, how long transcripts are handled, and how Cereal knows which lines are yours.

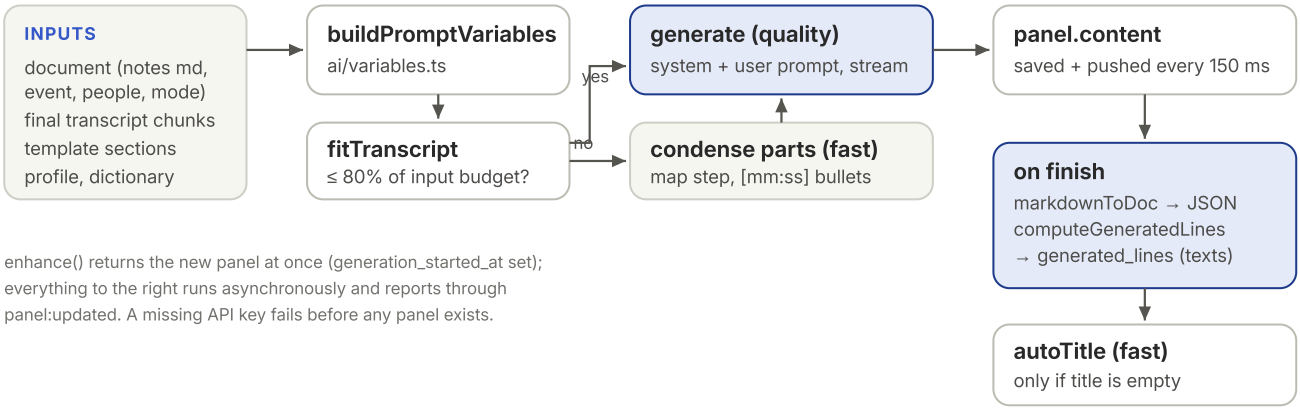


Figure 8.1 `AiService.enhance`, in `src/main/ai/service.ts`.

Panels

Each generation creates a `document_panels` row: one tab of "Enhanced notes", titled after its template. Its lifecycle is in the timestamps:

- `generation_started_at` is set on creation.
- `generation_finished_at` or `generation_failed_at` (plus `error`) is set at the end.

The renderer derives "generating" from those and shows the streamed Markdown *without its last, unfinished line*, so headings don't flash half-typed. Choosing a template on a note stores it in `selected_template`. Lecture notes default to the Lecture template; others default to Settings' choice, normally *Auto*.

The prompt variables

`buildPromptVariables` computes the same variable set Granola's client sends its backend. Granola renders it into prompts on the server; Cereal renders it into its own prompts in `ai/prompts.ts`.

Variable	How it is computed
`transcript`	`formatTranscript` of final chunks with prompt labels: "Ada (me)" (or "Me") and "Them"; "Speaker" for lectures.
`notes`, `headers`, `urls`	The notepad's Markdown; the user's own `#` headings; links found in the notes.
`participants`, `their_name`	"Name <email>" for the user and every attendee who isn't them; a natural-language list of the others.
`my_colleagues`, `external_attendees`	Attendees split by whether their email domain matches the user's.
`date`, `time`, `todays_date`	From the calendar event start, else the note's creation time.

Variable	How it is computed
`is_short_transcript`, `has_long_user_notes`	Transcript under 200 characters; 20 or more non-empty note lines. They switch extra instructions on.
`max_num_headings`	4 under 8k characters, 5 under 20k, ⌊len/10k⌋+1 under 80k, else 8.
`summary_headings`	`### Heading: description` per template section. Empty for Auto.
`user_dictionary`, `is_lecture`	Spelling list; lecture mode.

The prompt

The **system prompt** sets the job: rough notes plus transcript become shareable notes, or study notes in lecture mode. Its rules:

- Every point the user wrote appears, in their wording.
- Add the important points they missed: decisions, numbers, dates, commitments, open questions.
- Only what was said; leave out anything unclear rather than guess.
- Short, specific bullets.
- It explains the speaker labels honestly. "Them" is everyone else and is not separated, so don't attribute quotes to a specific person.
- It fixes the format: Markdown with `###` headings and `-` bullets, and no title or preamble.

The structure comes either from the template's headings, in order, or from an instruction to choose up to `max_num_headings` topical headings, keep the user's own headings, and end with *Next steps* (or *To review* for lectures).

The **user message** wraps the data in tags:

```
<meeting> Title · Date at time · Participants · Meeting type </meeting>
<my_notes> …the notepad's Markdown… </my_notes>
<transcript> [00:00] Ada (me): …  [00:07] Them: … </transcript>
[<current_notes> …edited enhanced notes… </current_notes>   only when rewriting]
Write the enhanced notes now.
```

Long meetings: a map step

Each provider declares `maxInputChars`, a rough character budget. If the transcript exceeds 80% of it, `fitTranscript` runs a map step:

1. Split the transcript on line boundaries into parts of at least 4,000 characters (half the budget).
2. Condense each part with the *fast* model into terse timestamped bullets that keep every decision, number, name and question.
3. Send the condensed parts as the transcript.

With Claude's 2-million-character budget this almost never triggers. With a small local Ollama model it is what makes enhancement possible at all.

Your lines in black, the AI's in grey

This is Granola's signature detail, and Cereal reproduces its mechanism exactly.

1. **Decide which output lines are the user's.** `shared/attribution.ts#computeGeneratedLines` normalises every line: it lowercases, strips list markers, Markdown and URLs, and tokenises.
   - An output *heading* is the user's only if it equals one of their headings.
   - Any other line is the user's if it contains at least 75% of the tokens of some user line. For very short user lines (two tokens or fewer) it must contain all of them and be at most three tokens longer.
   - Everything else counts as generated.
2. **Store texts, not positions.** `generatedLineTexts` converts the generated line indices into the trimmed *editor texts* of those lines, saved as `generated_lines`. Migration 5 converted older panels that stored indices.
3. **Colour by matching text.** A ProseMirror plugin (`shared/generatedLines.ts` + `components/editor/generatedLines.ts`) decorates every paragraph or heading whose trimmed text is in the set with `data-generated="true"`, and CSS turns those grey with a 0.1 s colour transition.

KEY IDEA

Because the match is on *text*, editing a grey line makes it stop matching and it turns black by itself, with no bookkeeping. Rewriting keeps the user's edits black too: the rewrite feeds the user's edited lines into the attribution as "user text".

Editing, rewriting, explaining

- **Editable panels.** When generation ends, the Markdown is converted once (`shared/markdown.ts#markdownToDoc`, line-oriented, one text block per line) into TipTap JSON (`content_json`). The panel then becomes an editor using the notepad's schema (`views/note/EnhancedEditor.tsx`). Edits save through `panels:update` on a 1,000 ms throttle that fires on both the first and the last change. Main stores the JSON and re-derives the Markdown (`docToMarkdown`), so search, copy and the agent see the edits. Saves are refused while the panel is generating, so a late save can't overwrite a rewrite.
- **Rewrite.** `rewrite(panelId, instructions)` aborts any running generation. It collects the panel's user-authored lines (lines not in `generated_lines`), resets the panel, and runs the same pipeline with the instructions appended to the system prompt and the current notes as `<current_notes>`. The chat's `rewriteMeetingNotes` tool instead creates a *new* panel with the extra instructions.
- **Auto-title.** If the note has no title, the fast model proposes one of at most six words.
- **Auto-enhance.** The *Enhance notes when transcription stops* setting (Settings → General) calls `enhance` from `stopRecording`.

Transcript Summary: "where did this line come from?"

Hover a paragraph in the enhanced notes and a small button appears. Clicking it runs `TranscriptSummaryService.start(requestId, documentId, line)`:

1. **Choose the transcript.** `selectTranscriptChunks` sends the whole final transcript when it fits 60% of the budget. Otherwise it picks the speaker turns sharing the most words with the line, plus two turns of context either side. Enhanced lines carry no timestamps, so there is nothing better to anchor on.
2. **Ask the fast model** (at most 700 tokens) for one to three plain sentences, with verbatim quotes on lines starting with `>`, or exactly `NO_CONTEXT`.
3. **Stream it.** The text so far (not deltas) goes out as `transcript-summary:update` every 80 ms. Nothing is stored.
4. **Link the quotes.** The renderer parses the reply into sentences and quote cards. `matchQuote` slides a word window over the chunks (score ≥ 0.6) to find which chunk a quote came from. Clicking a quote opens the transcript panel, scrolled to that chunk and flashing it.

GRANOLA

Granola runs generation on its own servers ○ :

- **The request.** The client posts a `prompt_slug` and the variables to `stream.api.granola.ai/v1/generate-summary`.
- **The stream.** Delimited JSON packets come back: `panel_id`, `completion` deltas, `generated_lines`, and a `ydoc` packet with the panel's Yjs state.
- **Collaboration.** The finished panel lives in a Yjs document, so several people can edit it (chapter 19).

Transcript Summary is `view-source-summary` through an LLM proxy, with quotes matched by Fuse.js.

CHAPTER 9

The LLM layer

Four providers with different APIs, one interface. The agent needs more from a provider than "complete this text", and this layer is where that difference is absorbed.

One interface, two calls

```
interface LlmProvider {
  id: 'anthropic' | 'openai' | 'gemini' | 'ollama'
  maxInputChars: number                              // rough budget before content must be condensed
  generate(opts: GenerateOptions): Promise<string>   // system + messages → streamed text (onText)
  step(opts: StepOptions): Promise<StepResult>       // ONE model turn with tools → blocks + stop reason
  listModels(): Promise<string[]>
  deleteFile?(remoteId: string): Promise<void>
}
```

- `generate` serves enhance, rewrite, titles, condensing, Transcript Summary and transcript compaction.
- `step` serves the agent. It sends the conversation so far plus tool specs, streams text and reasoning deltas through callbacks, and returns the assistant's *blocks*: `text`, `reasoning`, or `tool_call {id, name, args}`. It also returns a normalised `stopReason` (`end_turn | tool_use | max_tokens | refusal | pause | other`), the model name and token usage.

Every call names a **tier**. `quality` uses the main model (Claude Opus 5.5 by default). `fast` uses the cheap one (Claude Haiku 5.5) for titles, condensing, summaries and compaction.

The neutral transcript

The agent stores conversations in a provider-neutral form (`llm/provider.ts#ModelMessage`):

```
{ role: 'user',      content, attachments?: AttachmentRef[], messageId? }
{ role: 'assistant', blocks: ModelBlock[], replay?: { provider, content } }
{ role: 'tool',      results: [{ id, name, content, isError? }], note? }
```

Each provider translates this to its wire format: `toAnthropicMessages`, `toOpenAIInput`, `toGeminiContents`, `toOllamaMessages`. Tool specs are JSON Schema produced from the tools' zod schemas.

KEY IDEA: PROVIDER REPLAY

Modern models attach opaque state to their turns:

- Claude's **thinking blocks with signatures**.
- OpenAI's **encrypted reasoning items**.
- Gemini's **thought signatures** on function calls.

The next request must send them back *byte for byte*, or the API rejects it or loses the reasoning. So `StepResult.replay` carries the provider's native assistant content. It is stored in the transcript next to the neutral blocks, and replayed verbatim when the next turn goes to the *same* provider. If the user switches models mid-thread, the neutral blocks are used instead. Granola's tool calls carry the same idea as `providerReplayKey` plus provider metadata.

The four providers

Provider	How it is implemented
Anthropic	`@anthropic-ai/sdk` beta Messages streaming: • **Thinking:** adaptive (`thinking: {type: "adaptive", display: "summarized"}`) on models that support it, and `output_config.effort` low or medium by tier. • **Caching:** automatic prompt caching (`cache_control: ephemeral`). The tool list is kept in a fixed order so the cached prefix stays byte-stable. • **Tool input:** `eager_input_streaming` on tools. An unparsable tool input retries the step up to twice. • **Refusals:** the server-side fallback beta on supported models, and refusals surfaced as errors. • **Files:** images up to 256 KB inline (base64); larger images and PDFs through the Files API by id; other files as text `document` blocks.
OpenAI	The **Responses API**, with `store: false` so nothing is kept server-side. Reasoning models return encrypted reasoning items, which are replayed. Files go through the Files API as `input_file` and `input_image` with file ids.
Gemini	`@google/genai`. The model's parts are replayed part for part so thought signatures stay paired with their function calls. Files go through the Gemini Files API, which deletes them after 48 hours, so they are re-uploaded shortly before expiry.
Ollama	Plain HTTP to `/api/chat`, with NDJSON streaming and tool calls. Images are sent inline for vision models; other files are sent as their extracted text.

Choosing a provider and model

- **`providerConfig(settings, id?)`** resolves the provider, its two models and its key into a plain `ProviderConfig`. A missing key throws `LlmConfigError` with a message naming the exact variable and file.
- **`chatProviderConfig(settings, choice)`** applies the chat's model picker. `'auto'` means Settings' provider and model; `'anthropic/claude-sonnet-5-5'` means that model, keeping the provider's fast model for compaction. The choice is stored per thread (`chat_threads.model`) and per run (`agent_runs.llm`).
- **`createProviderFromConfig(config)`** needs only the plain config, not the settings service. That is how the agent's utility process builds providers: main resolves the config and sends it over in the run spec.
- **`chatModels()`** builds the model picker. It merges a curated list (`shared/models.ts`), whatever model Settings names, and Ollama's installed models (with a 1.5 s timeout). Providers without a key are listed but disabled, with the reason.

Files, uploaded once

Chat attachments reach providers through a `FileAccess` object: bytes, extracted text, and "my upload of this file" plus "remember my upload". `llm/files.ts#uploadOnce` checks the cache, deduplicates concurrent uploads of the same file through an in-flight map, uploads, and records the remote id. The record is saved in `chat_attachment_uploads`, so later turns and later runs reuse it. An upload within 5 minutes of expiry counts as gone.

GRANOLA

Granola's client never talks to a model provider. Everything goes through its backend: `chat-with-documents`, `generate-summary`, `llm-proxy-stream` `o` . The server also picks models (`get-chat-models`), with "Auto" as the default, and a workspace can lock the choice. Cereal's provider layer is the price of having no server, and the benefit: any key, any model, including local ones.

CHAPTER 10

The agent, part 1: the core

Cereal has two agent products: *Ask Cereal*, the chat, and the *Assistant*. They share one engine, built the way Granola builds its cloud "Keto" agent, but with Cereal's main process playing Granola's servers.

Vocabulary

Six nouns explain the whole subsystem. Learn these first.

Noun	What it is	Stored in
Thread	A conversation. A chat thread belongs to one note, or to "all notes". An Assistant thread is the one *conversation* or one of its *tasks*.	`chat_threads`, `agent_threads`
Message	What the *user* sees: a question, or an answer made of `outputs` (reasoning, tool calls, text) plus status.	`chat_messages`, `agent_messages`
Model transcript	What the *model* sees: the neutral `ModelMessage` turns of a thread, append-only, split into *epochs* when compacted.	`agent_model_messages`
Run	One execution of the loop answering one input. Kind `chat` or `assistant`; a context (`note-chat`, `global-chat`, `conversation` or `task`) that decides its tools; a status.	`agent_runs`
Event	A small fact a run reports as it goes: `message.delta`, `tool.started`, `approval.requested`, `run.status` and so on. Folding events builds a message's outputs.	`agent_events` (Assistant runs)
Approval	A tool call held until the user says yes or no.	`agent_approvals`

KEY IDEA: TWO RECORDS OF ONE CONVERSATION

The *message* and the *model transcript* are deliberately separate. The UI needs a friendly, mutable record: a label like "Searched meetings", a status, an undo flag on an edit. The model needs an exact, immutable record: the tool call ids, raw results and provider replay blocks it produced. Keeping both lets each be right for its reader. Edit and resend, compaction and approval resumption all operate on the transcript, never on the messages.

The split: runtime and backend

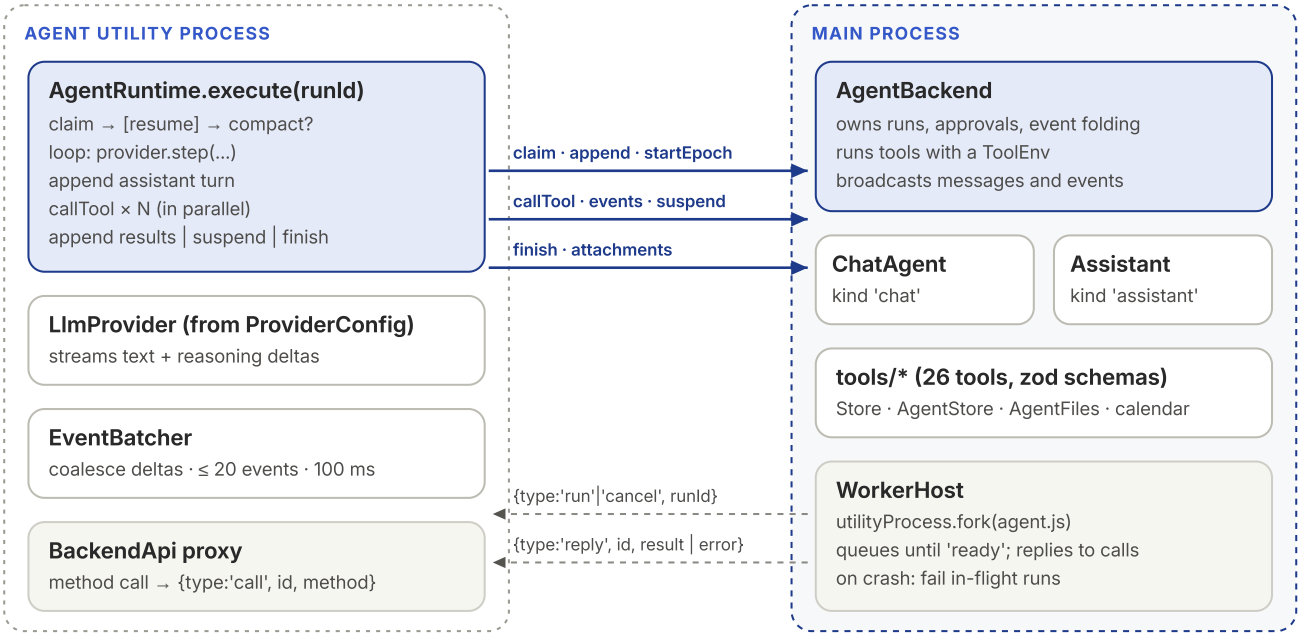


Figure 10.1 The runtime decides what to do next; the backend does it. Every arrow on top is an async call over the utility process's parent port. Tests swap `WorkerHost` for `InProcessHost`, which runs the same `AgentRuntime` in the same process.

The model/tool loop (`agent/runtime.ts`) runs in its own Electron utility process. It holds no database handle and no tools. Everything with a side effect is a call back to the backend in main (`agent/backend.ts`). The contract is `agent/protocol.ts#BackendApi`:

Action	What the backend does	Keto (Granola) analogue
`claim(runId)`	If the run is `queued`, mark it `running` and return a `RunSpec`: provider config, system prompt, tool specs, the thread's transcript, `maxSteps`, any resume information, and known provider uploads. Otherwise return null.	`worker.next` / `claim`
`appendTranscript`	Append turns to the thread's current epoch.	`assistant.model.append`
`startEpoch`	Begin a new epoch holding a summary plus kept turns.	(context reset)
`callTool(runId, call, approved?)`	Validate, check the policy, then run the tool or create an approval. Returns `{kind:'result'}` or `{kind:'approval_required'}`.	`assistant.approval.request` + tool execution
`events(runId, events)`	Fold the events into the message, persist, broadcast.	`events` (1–20 per call)
`suspend(runId, pending)`	Park the run as `waiting` with the step's partial results.	approval wait
`finish(runId, outcome)`	Settle the run and its message: completed, failed or cancelled.	`finish`
`attachmentBytes|Text`, `recordUpload`	Give a provider a chat file, checking it belongs to the run's thread, and cache the upload.	—

The worker side (`agent/worker/entry.ts`, 41 lines) is small. A `Proxy` turns any method call on `backend` into a `{type:'call', id, method, args}` message to the parent port, and resolves or rejects the matching promise when `{type:'reply', id}` comes back. `WorkerHost` on the main side forks the process lazily on the first run, queues messages until the worker says `ready`, routes calls to the backend's methods, and tracks in-flight runs. If the process exits, every in-flight run is failed with "The agent process stopped unexpectedly", and the next run starts a fresh worker.

WHY THIS SHAPE (GRANOLA)

Granola's Assistant ("Keto") runs as a per-user cloud worker: an AWS ECS task with an EFS filesystem `I`. It talks to Granola's backend through an action protocol `O`:

- `claim`, `assistant.model.append`, `events`, `finish`;
- `inbox.next` and `inbox.ack`;
- `assistant.approval.request`.

Cereal keeps the shape and moves it on-device: the utility process is the worker, and main is the backend. The payoff is the same as Granola's. The loop is stateless between calls, every effect is persisted the moment it happens, and a run can stop (crash, quit, approval wait) and resume from the database.

Run states

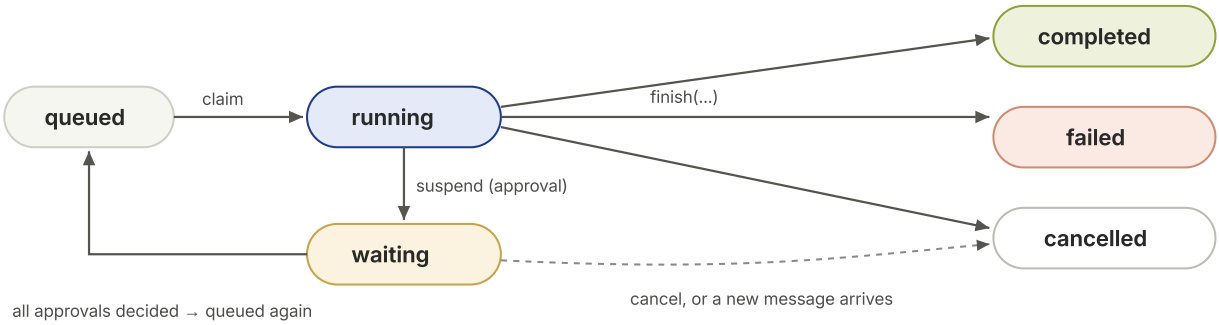


Figure 10.2 A run's statuses. `queued`, `running` and `waiting` are "active"; a thread has at most one active run. On startup `recover()` fails runs left `queued` or `running` by a quit, but keeps `waiting` runs, so an approval can be answered days later.

The loop, step by step

This is `AgentRuntime.execute` with the error handling simplified:

```
spec = await backend.claim(runId)                  // null → someone else has it, or it was cancelled
provider = createProvider(spec.llm)
messages = spec.transcript

if (spec.resume) {                                   // coming back from an approval wait
  results = spec.resume.pending.results              // calls that already ran in that step
  for call of spec.resume.pending.waiting:
    results += decision == approved ? await backend.callTool(runId, call, approved=true)
                                    : "The user declined this action. Do not try it again…"
  messages = append({ role:'tool', results })
}
messages = compact(messages, budget = 60% of provider.maxInputChars)   // may start a new epoch

for step in 0 ..< spec.maxSteps:                     // 12 chat · 16 conversation · 30 task
  result = await provider.step({ system, messages, tools, onText → message.delta, onReasoning →
reasoning.delta })
  emit model.usage; flush events
  if result.stopReason == 'refusal': throw
  messages = append({ role:'assistant', blocks: result.blocks, replay: result.replay })
  calls = tool_call blocks
  if no calls: break                                 // the model answered: done
  if result.stopReason == 'max_tokens': throw        // never run a tool call whose JSON was cut off
  if last step: answer calls with "Not run: step limit reached"; break
  replies = await Promise.all(calls.map(c => backend.callTool(runId, c)))   // tools run in parallel
  if any reply is approval_required:
    await backend.suspend(runId, { results: the finished ones, waiting: the held calls })
```

```
    return                                          // the process is free; resume comes later
  messages = append({ role:'tool', results, note: step == maxSteps-2 ? STEP_LIMIT_NOTE : undefined })
await backend.finish(runId, { status: 'completed' })
```

Invariants the loop protects

- **The transcript never ends on unanswered tool calls.** Every provider rejects a history where an assistant tool call has no result. The runtime tracks `unanswered` calls. If the run fails or is cancelled mid-step, it appends error results ("Stopped by the user." or "Not run: the turn ended early.") before finishing. `cancelRun` does the same for a run parked in `waiting`.
- **The step limit is soft, then hard.** One step before the limit, the tool results carry a note: *answer now with what you have*. On the last step, tool calls are answered "Not run", and the message gets "(Stopped after reaching the step limit.)".
- **Partial JSON never executes.** A `max_tokens` stop with tool calls is an error, because a truncated argument object can still parse as valid but wrong.
- **Pause turns continue.** A `pause` stop with no calls (a provider's long server-side turn) loops again without user input.

Events: how answers stream

The runtime never builds the message itself; it emits `AgentEvent`s (`shared/agent.ts`), named after Keto's:

```
run.status {status, error?}          message.delta {messageId, delta}        reasoning.delta {messageId,
delta}
tool.started {toolCall}              tool.completed {toolCall}               model.usage {model,
tokens…}
approval.requested {toolCallId, approval, label, detail}                     approval.decided
{toolCallId, approval}
```

Three layers handle them.

1. **EventBatcher** (runtime). It merges consecutive deltas of the same kind for the same message, and sends a batch when it reaches 20 events or 100 ms. It chains sends on a promise, so batches arrive in order.
2. **AgentBackend.emit** (main). It folds each event into the run's live outputs with `applyAgentEvent`, the *same pure reducer* the renderer uses:
   - Text deltas extend the last text output, or start one.
   - Tool events upsert by tool-call id.
   - Approval events patch the call.

   Then it persists:
   - **Chat runs:** the `chat_messages` row is updated, and a `chat:message` push is sent at most every 150 ms.
   - **Assistant runs:** each event is appended to `agent_events` with a per-run `sequence`. The message's outputs are saved with `applied_sequence` (the highest sequence folded in), and every record is broadcast as `agent:event`.
3. **The renderer.** The chat replaces its message on each `chat:message`. The Assistant view folds `agent:event`s into its messages, using `applied_sequence` to skip anything it already has.

KEY IDEA: CATCHING UP WITH AFTERSEQUENCE

When the Assistant page opens mid-run, it loads the thread's messages. For each message still `in_progress`, it asks `agent:events(runId, message.applied_sequence)` for what it missed, folds those, and then follows live events. Duplicates are harmless because folding is keyed by sequence. This is exactly how Granola's `keto-socket` works: a POST `{runId, afterSequence}` returning SSE frames, with duplicates dropped by sequence. Events older than 14 days are pruned at startup.

Message outputs

A finished answer is an ordered list of outputs:

```
[ { type:'reasoning', text:'The user wants last week…' },
  { type:'tool_call', id:'toolu_1', name:'listMeetings', label:'Listed meetings', args:
{time_range:'last_7_days'},
    completed:true, result:{count:5, total:5}, startedAtMs, completedAtMs },
  { type:'tool_call', id:'toolu_2', name:'draftEmail', …, result:{kind:'email', to:[…], subject, body}
},
  { type:'text', text:'Here is a summary of last week [1][3]…' } ]
```

`components/AgentOutputs.tsx` renders them:

- **Reasoning** as a collapsible block.
- **Read-only tool calls** grouped into one compact activity row ("Searched meetings · Read 3 meetings").
- **Drafts, edits, tasks, results and approvals** as cards.
- **Text** as Markdown with clickable `[n]` citations.

`content` holds the joined text outputs, for search, copying and history. On `finish`, the backend marks still-open tool calls "Stopped". It also sets "No response from the model." when a completed run produced nothing, and extracts citations (chapter 11).

CHAPTER 11

The agent, part 2: tools, approvals and the chat

A model is only as useful as its tools. This chapter covers how a tool is declared, which tools exist where, how a tool call is held for approval, how answers cite meetings, and how the chat assembles each turn.

Declaring a tool

```
export const readTranscript = defineTool({
  name: 'readTranscript',
  description: 'Read the transcript of one meeting, with [mm:ss] timestamps. Long transcripts come in pages…',
  input: z.object({
    meeting_id: z.string(),
    offset: z.number().int().min(0).default(0).describe('First line to return.'),
    limit: z.number().int().min(50).max(2000).default(600)
  }),
  label: { active: 'Reading transcript', done: 'Read transcript' },   // what the UI shows
  contexts: ['note-chat', 'global-chat', 'conversation', 'task'],     // where it is offered
  execute(args, env) { …; return { content: '…text for the model…', data: { lines, total } } }
})
```

- **One schema, two uses.** The zod `input` validates arguments at runtime. It is also converted once, and cached, into the JSON Schema the model sees (`toolSpec`, using zod's `io: 'input'` view so fields with defaults are optional to the model).
- **Two outputs.** `content` goes to the model. `data` stays on the tool call for the UI, for example the email draft that becomes a card.
- **Optional fields.**
  - `policy`: `'allow'` or `'ask'`, or a function of the arguments.
  - `approval(args)`: what the approval card says.
  - `affects`: notes, memory, tasks or routines.
  - `clientApplied`: the renderer performs the effect.
- **Bad arguments.** They are not exceptions to the run; they are answers to the model. `parseArgs` returns *"Invalid arguments for readTranscript: offset: Too small..."*, or for unparsable JSON `{"INVALID_JSON": …}`, as an error result. The model reads it and retries.
- **A stable order.** `tools/index.ts` lists every tool in a fixed order, because the tool list is part of the prompt prefix that Anthropic's cache keys on.

The tool catalogue

Tool	Contexts	What it does
`searchMeetings`	all	FTS5 search with optional people and date filters; returns numbered meetings with snippets.
`listMeetings`	all	Meetings in a time range (today … last 90 days, all) or a folder.
`readMeetings`	all	Up to 10 meetings in full: notes (8k), latest enhanced notes (12k), attendees, transcript length.
`readTranscript`	all	A transcript page of 600 lines by default, with paging hints.

Tool	Contexts	What it does
`getUpcomingCalendarEvents`, `searchCalendar`	all	Synced Google Calendar events, with the linked note's id when there is one.
`listFolders`	all	Folders with their ids and sizes.
`readSkill`	all	Load a `SKILL.md` playbook's instructions.
`draftEmail`, `draftCalendarEvent`	all	Return a draft as `data`; the UI shows a card with Copy, `mailto:`, or a prefilled Google Calendar page. Nothing is sent.
`editMeetingNotes`	note-chat	Replace an exact passage of the user's notes. Checked in main, applied in the editor, can be undone.
`rewriteMeetingNotes`	note-chat	Start a new enhanced-notes panel with instructions.
`list_files`, `read_file`	conversation, task	Browse the agent folder.
`write_file`, `edit_file`	conversation, task	Allowed inside `memory/`; **ask** anywhere else. Edits must match exactly once.
`startTask`, `messageTask`, `stopTask`, `readTaskResult`, `listTasks`	conversation	Delegate work to parallel task threads and manage them.
`publishHome`	conversation	Replace the Assistant home: a briefing plus tasks grouped under headings.
`createRoutine`	conversation, task	**Ask**: schedule a message to the agent with `at`, `rate` or `cron`.
`listRoutines`, `deleteRoutine`	conversation, task	Manage routines.
`publishResult`	task	Finish a task: `done`, `needs_input` or `failed`, with a summary and a result.

Granola's Ask chat has 82 named tools, plus MCP and connector tools, and they run on its servers (chapter 23). Cereal's 26 cover the local subset: meetings, calendar, drafts, note edits, files and orchestration. There is no web search, email, Slack, CRM or MCP.

The ToolEnv

Tools run in main with a `ToolEnv`:

- the stores and the settings snapshot;
- the run;
- `documentId`, the note a note-chat is about;
- `calendarBetween`;
- `rewriteNotes`, which calls `AiService.enhance`;
- `assistant`, the `AssistantOps` that task tools call;
- `emit`, to report extra events such as `memory_changed`.

When no Assistant is wired, as in chat-only tests, `assistant` is a `Proxy` that throws "The Assistant is not available.".

Citations: stable [n] numbers

Every time a tool shows the model a meeting, it calls `cite()`, which asks `AgentStore.sourceNumber(threadId, documentId, title)` for that meeting's number *within this thread*. The first meeting seen is [1], the next new one [2], and so on, stored in `agent_sources`. A note's own chat always numbers the open note [1]. The prompt says: *cite with the bracketed number right after the sentence; only cite numbers you were given.* When a run finishes, `extractCitations` scans the answer for `[n]` markers and stores the matching `{n, document_id, title}` list on the message. The renderer turns them into links. Because numbers are per thread and never reused, "[3]" means the same meeting in every answer of the conversation.

Approvals

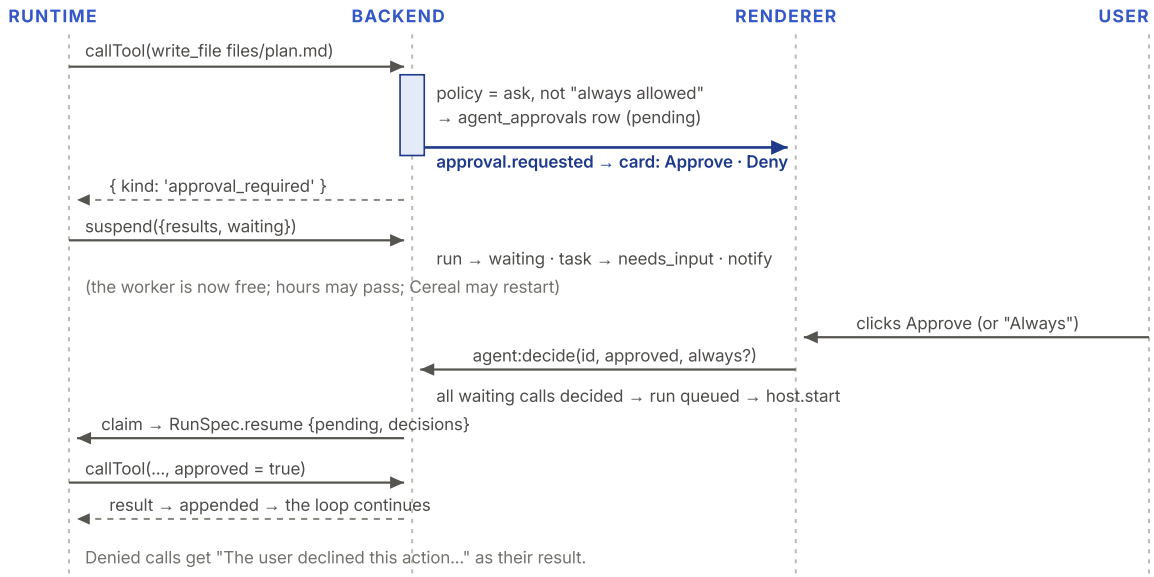


Figure 11.1 A tool call held for approval. Granola's approval flow has the same shape: the backend answers `allow`, `approval_required` or `denied`.

- **What asks.** Only two policies ask today: writing or editing files outside `memory/`, and creating a routine. The machinery is generic, so a future "send email" tool would set `policy: 'ask'`.
- **Always allow.** It adds the tool name to `settings.assistant.alwaysAllow`, and later calls skip the question.
- **A new message declines.** Sending a new message while a step waits cancels the run, so the waiting calls are declined in the transcript, as in Granola's chat.
- **Persistence.** The pending step lives in `agent_runs.pending` and the decisions in `agent_approvals`. The resume is exact, even across restarts.

Epochs, compaction, and edit-and-resend

A thread's transcript is **append-only**: turns are inserted with increasing `sequence` and never edited. The reason is provider replay. A Claude thinking block is only valid after the exact history that produced it, so rewriting history would invalidate the replayed reasoning.

Two operations still need to "change" history. Both start a new **epoch** instead; the old epoch stays in the table.

- **Compaction** (`runtime.ts#compact`). Before the loop, if the transcript exceeds 60% of the provider's budget:
  1. Walk back from the end to find a user turn such that the kept tail is at least half the budget.
  2. Summarise everything before it with the fast model. The prompt keeps every fact, decision, meeting title with its [n], meeting_id, task_id, routine_id and file path.

3. Start a new epoch: *(summary)* plus the kept turns, with their provider replay *stripped*, because their history changed.

- **Edit and resend** (`chat.ts#rewind`). Editing a sent question starts a new epoch containing the transcript *before* that turn, unchanged so replay stays valid. If those turns were compacted away, it is rebuilt from the earlier chat messages. The chat messages from the edited one onward are deleted. Files the edited message keeps stay bound to its reused id, and every other file is removed. `retry` is "edit and resend the last question, unchanged".

The chat: `agent/chat.ts`

`ChatAgent.send(input)` runs synchronously in main and returns `{threadId, messageId}` before any model call:

1. **Validate.** Some text or files are required. The chosen model must be configured, which raises a friendly error otherwise. Files must be ready, and only drafts (or the edited message's own files) may be sent.
2. **The thread.** Reuse it or create one, titled with the first 80 characters for now, and save the model choice on it. If a run is active: decline it if it is waiting, cancel it if this is an edit, otherwise refuse with "Wait for the current answer to finish".
3. **Seed old threads.** Threads from before the agent existed get a transcript built from their messages.
4. **Save the question.** Store the user message with its context (mentions, selected text, response style), bind its files, and create the empty assistant message (`in_progress`). Both are pushed to the UI immediately.
5. **Build the model turn.**
   - *Preamble:* mentioned notes with their [n] numbers and ids, and the selected text in `<selected_text>`.
   - *The question:* a leading `/recipe-slug` is expanded into the recipe's prompt, and its use is counted.
   - *Optionally:* "give a more detailed answer" (*Say more*).

   Files ride along as `AttachmentRef`s.
6. **Start.** Append the turn to the transcript and create a run (`note-chat` or `global-chat`).

`prepare()`, called by `claim`, builds the system prompt (`agent/prompts.ts#chatSystemPrompt`):

- **In a note's chat,** the note goes into the prompt: notes, latest enhanced notes, and the transcript if it fits 40% of the budget, otherwise a pointer to `readTranscript`.
- **In the global chat,** the prompt has the note count, the eight most recent meetings, and instructions to look things up rather than answer from memory.
- **Both** get today's date and timezone, the user profile, the citation and draft rules, and the skill list.

Prompts are stable within a day, so providers can cache them; anything that varies per message goes into the user turn. After a thread's first answer completes, `finished()` asks the fast model for a six-word title and pushes `chat:thread`.

Edits the renderer applies

`editMeetingNotes` is the one tool whose effect happens in the UI. Granola does the same, because the open editor owns the document and an undo must be an editor transaction.

1. **Check in main.** The backend's `execute` only checks that `textToReplace` occurs in the note's plain text, and returns "sent to the editor". The tool call is marked `applied: null`.
2. **Apply in the note.** `NoteView` watches the thread's messages. For each completed, not-yet-handled `editMeetingNotes` call, `lib/noteEdits.ts#applyNoteEdit` finds the passage in the TipTap document, mapping plain-text offsets to ProseMirror positions across block boundaries. It replaces the passage, or appends when it's empty, and selects the new text.
3. **Report back.** The renderer reports `chat:tool-applied {applied: true|false}`.
4. **Undo and Reapply.** The edit card's Undo applies the reverse edit and sets `reverted: true`; Reapply flips it back.

GRANOLA

Granola's chat is `POST stream.api.granola.ai/v1/chat-with-documents` `O` :

- **Request:** `chat_history`, a `chat_context` (meeting, live-meeting, global, list, search, view, shared-with-me), `document_ids` and a `prompt_config`.
- **Response:** delimiter-separated JSON chunks: `text_delta`, `output_delta`, `context_sources_delta`, `stream_completed`.
- **Citations:** structured `{meeting_id, citation_source_type, citation_source_quote}` per response line.
- **Before sending:** pending approvals are auto-denied when a new message is sent.

Cereal's chat reproduces the outputs, tool-call fields and approval semantics, with the loop local.

CHAPTER 12

The agent, part 3: the Assistant

The chat answers a question. The Assistant runs work: it delegates to tasks that run in parallel, wakes itself on a schedule, asks for help, remembers things and follows playbooks. It is built on the same runs, but adds threads, an inbox and a clock.

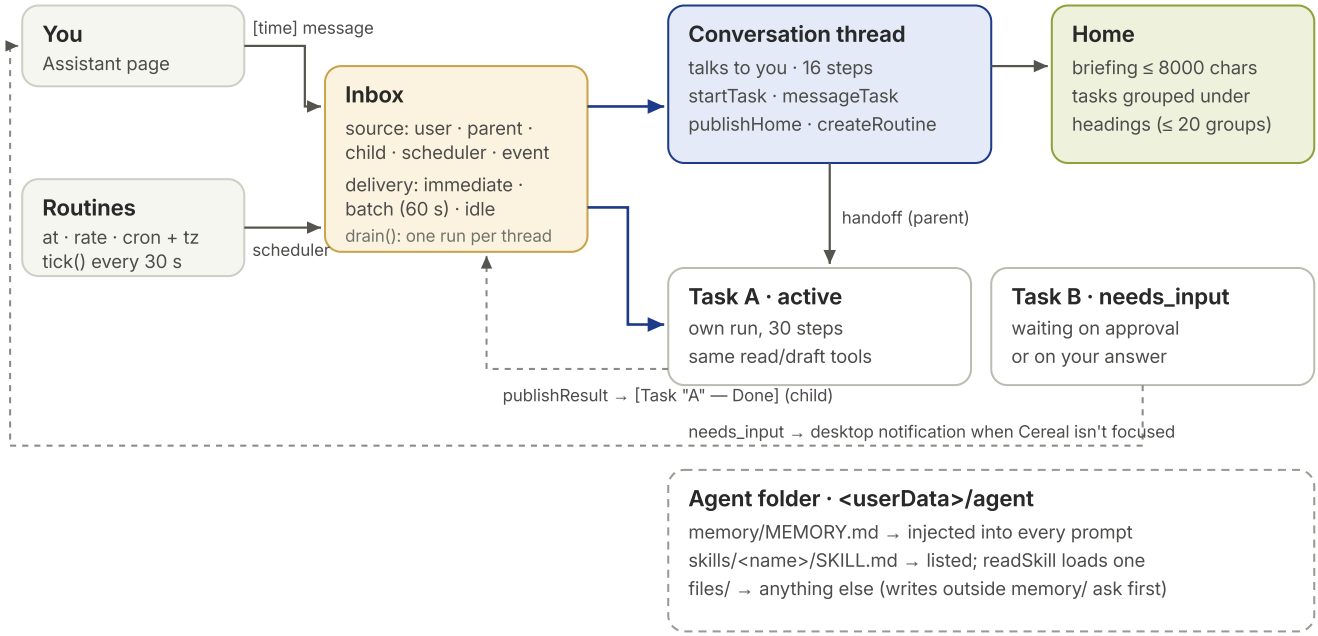


Figure 12.1 The Assistant's moving parts. Every arrow into a thread goes through the inbox; every run is an ordinary agent run of kind `assistant`.

Threads

- **One conversation.** `AgentStore.conversation()` returns the current, non-archived conversation thread, creating it on first use. *Reset* archives it with its tasks and clears the Home.
- **Tasks** are child threads (`parent_thread_id` = the conversation). Each has:
  - `title`;
  - `status`: `active`, `needs_input`, `done`, `failed` or `cancelled`;
  - `lifetime`: `once`, or `ongoing` for tasks that keep receiving messages and routine runs;
  - an optional `follows_thread_id` (a task that continues another);
  - `summary` and `result_message_id`;
  - `pending_approval`.
- **Message authors** are `user`, `agent` or `parent`, the last being the conversation handing a task its instructions. Inputs from tasks, routines and events reach the model but are not shown as messages, as in Keto.

The inbox and drain()

Nothing starts a run directly. Everything enqueues an **inbox item**: your message, a task's handoff, a task's result reported upward, a routine firing. Then `drain()` runs:

1. Find threads with deliverable items (`AgentStore.readyInboxThreads`):
   - `immediate` items are always deliverable;

- `batch` items once their window passes (default 60 s; then *every* item of that thread goes together);
- `idle` items only when no Assistant run is queued or running anywhere.

2. Skip threads that already have an active run: **one run per thread**, but many threads in parallel.
3. Take all the thread's items, join their texts, and append them as one user turn.
4. Create the agent's reply message (`in_progress`), mark a task `active`, and create a run with context `task` or `conversation`.

`drain()` runs after every enqueue, after every run finishes (`finished()` hook), when the Assistant is switched on, and on every `tick()`. Items are labelled so the model knows who is speaking:

```
[Mon 14:05] Can you prep me for the Acme call tomorrow?                          ← you (user)
[From the conversation, Mon 14:06] Use their last two meetings, not the intro call. ← messageTask (parent)
[Task "Acme prep" (task_id 4f…) — Done] Brief ready: 3 open issues, …            ← a task reporting (child)
[Routine 9c…: Weekdays at 09:00, Tue 09:00] Summarise yesterday's meetings.      ← the scheduler
```

A task's life

1. **Start.** The conversation calls `startTask {title, handoff, lifetime, follows_task_id?}`. The backend creates the task thread and shows the handoff as a `parent` message. It enqueues the handoff, plus the earlier task's result when it follows one. The conversation's prompt insists the handoff must be complete, because *the task cannot see the conversation*.
2. **Work.** The task runs with the task prompt and up to 30 steps. It reads meetings, drafts, writes files. Approvals put it in `needs_input` with `pending_approval` set, and send a notification if Cereal isn't focused.
3. **Report.** The task must end by calling `publishResult {status, summary, text}` once. `settle()` stores the status, summary and result message, then enqueues a `child` item to the conversation: `[Task "title" (task_id …) — Done] summary + text`. The conversation then runs and tells you in a sentence or two. If a run ends *without* publishing, `finished()` settles it anyway: the last text becomes a `done` result, a failure becomes `failed`, a stop becomes `cancelled`.
4. **Follow up.** `needs_input` means the conversation asks you; your answer goes back with `messageTask`. Ongoing tasks keep receiving messages and routine firings.

`publishHome` keeps a briefing and grouped task list current. It is stored in the settings table under `agent.home`, with unknown task ids filtered out, and shown on the Assistant page.

Routines

Routines (`agent_schedules`) are messages the agent sends itself on a schedule. `createRoutine` requires your approval. The grammar (`agent/schedule.ts`) follows Granola's Assistant:

Expression	Meaning
`at(2026-10-21T09:00)`	Once, at that wall-clock time in the routine's timezone, or at an explicit offset if one is given.
`rate(15 minutes)`	Every N minutes, hours or days. The minimum is 5 minutes.
`cron(0 9 * * 1-5)`	Five fields; names allowed (`mon-fri`, `jan`); 0 or 7 is Sunday. The AWS six-field form with year `*` is accepted.

`nextFireAfter(expression, after, timezone)` computes the next instant:

- **For `at`**, the wall-clock time is converted to an instant with a short fixed-point iteration over `Intl.DateTimeFormat`. It is correct across daylight-saving changes, without a timezone library.

- **For `cron`**, it walks forward minute by minute in the target timezone, but skips whole days or hours that can't match. Day-of-month and day-of-week combine with OR when both are restricted, like classic cron.

Every 30 s, `tick()` delivers each *due* routine to its task (if that task isn't cancelled) or to the conversation, records `last_fired_at`, and computes the next time. A routine whose time passed while Cereal was closed fires **once** on the next tick, not once per missed slot. One-shot routines disable themselves.

Memory, skills and the sandboxed folder

- **The folder.** `AgentFiles` roots everything at `<userData>/agent` with three subfolders: `memory/`, `skills/` and `files/`. `resolve(path)` rejects absolute paths and NUL bytes. It then checks containment twice: *lexically* (no `..` escape), and against the *real path* of the nearest existing ancestor, so a symlink inside the folder can't point outside. Files are capped at 1 MB; reads return 60k characters per call.
- **Memory.** `memory/MEMORY.md` plus the names of the other memory files (`memoryIndex`, about 8k characters) is injected into every Assistant prompt inside `<memory>`. The prompt tells the agent to keep it up to date, one short line per durable fact, and never to store secrets. Writes inside `memory/` need no approval.
- **Skills** are `skills/<name>/SKILL.md` playbooks with frontmatter, validated like Granola's skill library:
  - `name`: kebab-case, at most 64 characters, equal to its folder;
  - `description`: 1–1024 characters.

  Valid skills are listed in prompts ("load one with readSkill before doing the work it covers"). Invalid ones are reported in the Skills page. A starter skill, `meeting-follow-up`, is seeded on first run.
- **Your instructions.** Standing instructions from Settings → Assistant are appended to every Assistant prompt.

A DELIBERATE DEVIATION

Granola's cloud workers get a `bash` tool in a container. Cereal's agent runs on your desktop, so it gets no shell at all, only four file tools confined to the agent folder. That is the single most important safety property of the Assistant.

GRANOLA: SIX AGENT SURFACES

Granola actually ships six agent-like systems `o`, all running server-side:

- **Ask chat** (`chat-with-documents`).
- **Keto, the Assistant**: `keto-gateway` actions plus `keto-socket` SSE.
- **Buddy/Hermes**: a separate `raisin` backend with NDJSON packets, Telegram pairing and cron jobs. It looks like a hosted open-source agent runtime `I`.
- **Granny voice calls**: WebRTC to `granny-live`, delegating long work to the text agent.
- **Dispatch**: post-meeting drafts for email, Slack, Linear and calendar.
- **A reminder agent** with its own card window.

Cereal deliberately puts its two surfaces on one local core. The surfaces differ in transport and UI, not in the loop. `docs/granola-agents.md` has the full catalogue.

CHAPTER 13

The interface

The renderer is a React 19 single-page app that follows Granola's layout and motion closely. Its job is to show state that lives in main and to send intentions back. It owns very little data itself.

Boot and global wiring

- **Boot.** `main.tsx` loads the bundled fonts (Inter at optical size 28, and IBM Plex Serif for display text, both Granola's own declared fallbacks), the stylesheet and `<App/>`.
- **On mount,** `App.tsx`:
  - loads settings, documents, folders, templates, upcoming events and the recording state;
  - resolves the theme and tells main through `window:chrome`, so the title-bar overlay's colours match;
  - installs the global shortcuts: `Ctrl K` search, `Ctrl N` new note, `Ctrl ,` settings, `Ctrl S` sidebar;
  - subscribes to the app-wide pushes: recording state, levels, transcript chunks, panel updates, document, folder, calendar and settings changes, and `navigate`.

Routing and state

- **Routing** is a 50-line hash router (`lib/router.ts`): `useSyncExternalStore` over `hashchange`, and `parseRoute` into a typed union (`home`, `note`, `folder`, `chat`, `assistant`, `recipes`, `templates`, `settings`). Hash routes work with `file://` pages, and main can navigate the window by pushing `navigate {route}`, for example from a notification click.
- **Global state** is three small zustand stores:
  - `useApp` (`store.ts`): cached lists, settings, recording state and levels, plus per-note transcripts and panels, merged with the same shared reducers main uses.
  - `useLayout`: sidebar width and open state, saved to `localStorage` with a 300 ms debounce, with narrow-window behaviour.
  - `useUi`: search palette and templates modal.
- **Local state.** Anything else lives in the component that needs it.

> **KEY IDEA**
>
> The renderer's stores are *caches* of main, not sources of truth. A push either carries the new object or tells the view to re-fetch. If you are unsure whether data is current, ask main again; it costs a microsecond.

The shell

- **Frameless window.** `titleBarStyle: 'hidden'` plus a 44 px `titleBarOverlay`. The toolbar is the drag area. Its right-hand spacer is sized with `env(titlebar-area-*)`, because GNOME's overlay shows only a close button, not Windows' three.
- **Sidebar** (`components/shell/Sidebar.tsx`):
  - Home, Assistant, Chat, then "My notes" folders, then footer icons and an account menu.
  - Resizable from 192 to 330 px.
  - It closes itself on note pages and restores your choice when you leave.
  - Under 768 px it becomes a floating overlay.
- **Toolbar** (`Toolbar.tsx`): pages render their own controls into it through a portal (`<ToolbarContent>`), and content scrolls under a blurred fade.

- **Overlays:** the Ctrl+K search palette (FTS5 hits with «» snippets) and the two-pane templates dialog.

The note page: `views/note/NoteView.tsx`

- **Header.** The title, then an options row of chips (`MetaChips`: date, mode, folder, template). *My notes* / *Enhanced* tabs appear once a panel exists; the Enhanced tab's menu switches, regenerates and deletes panels.
- **My notes** is `components/Notepad.tsx`, a TipTap editor: StarterKit with H1–H3, links and task lists.
  - Every change produces the ProseMirror JSON plus plain text and Markdown (`shared/markdown.ts`).
  - `NoteView` coalesces edits and saves them 500 ms after the last keystroke through `documents:update`.
  - Ctrl+click opens links.
- **Enhanced** is `EnhancedView`. While generating, it shows the stream with a colour-drifting "generating" label. Afterwards it shows `EnhancedEditor`, with grey AI lines and Transcript Summary popovers (chapter 8).
- **The bottom bar.** One centred row holds the `RecordingWidget` (Start, Stop or Resume, with live dancing bars) and the chat input. Clicking the widget opens the `TranscriptPanel` above it in the same card: live bubbles, follow-to-bottom, Find, Copy. Typing in the chat expands the chat panel instead. A *Generate notes* or *Enhance notes* pill rises above the bar once there is something to enhance.
- **Agent edits.** The page applies the chat's `editMeetingNotes` calls to its editor and offers Undo (chapter 11).
- **Share menu.** *Copy as Markdown* and *Copy transcript* only. There is no link sharing (Part III).

The chat surfaces

Surface	Where	Notes
`FloatingChatPanel`	Note pages	The bar's panel grows upward, in state `idle`, `chat` or `transcript`. Resizable height, remembered; spring animation.
`ChatSidebar`	Home, folders	A docked panel that floats on narrow windows; history menu, new chat, maximise.
`ChatView`	The Chat page	Full-page chat across all notes, with a greeting, recent threads and recipe chips.

All three use `useChatSession(documentId)` for threads and messages, kept live from `chat:message` and `chat:thread`, and the same `Composer` (`components/chat/Composer.tsx`, about 700 lines). The Composer is a TipTap editor with:

- inline **recipe** (`/slug`) and **mention** (`@Title`) atoms, with their menus (`extensions.ts`, `Menus.tsx`, fuzzy ranking in `shared/chatbar.ts`);
- context **chips** for mentions, selected text and files;
- **attachments** by picker, paste or drag-and-drop. `useAttachments` validates them, resizes images over 1568 px in a canvas, and turns long pastes over 5,000 characters into a `.txt`;
- the **model picker**, **dictation** (Ctrl+T), and one round button that sends, stops or dictates.

The oats design system

`components/oats/` wraps `@base-ui/react` primitives (Menu, Select, Tooltip, Dialog, Popover, Switch) with Granola's measured classes, timings and springs. It also has:

- **Buttons and shapes:** `Button` variants, `MorphButton` (a pill that springs to fit new content), `NestedButton` and `SegmentedTabs`.
- **The toaster**, built on sonner.

- **`DancingBars`**, drawn with WebGL (`barsRenderer.ts`: capsules with motion blur, 60 fps only while moving) over invisible DOM bars that act as the layout template and the fallback.

Tokens live in `styles.css` as `--oats-*` variables: warm off-white surfaces, olive-black hairlines and Granola's type scale. The one change is the brand colour, which is Cereal's deep navy `#1E3A8A` instead of Granola's olive green. `docs/granola-ui.md` has every measured value.

CHAPTER 14

Widget, calendar and attachments

Three smaller subsystems, each with one hard problem: a window that must stay on top under Wayland, OAuth without a server, and files that must reach four different model APIs.

The floating widget ("nub")

The widget is a 34 px pill at the right edge of the screen. It appears while Cereal records and the main window isn't focused, or when a detected meeting is waiting for an answer. `app.ts#updateWidget` decides:

```
show = widgetState.mode === 'prompt' || (recording && !mainWindowFocusedAndVisible)
```

Its state is `{mode: 'recording', documentId}` or `{mode: 'prompt', title, app}`. Its actions are `open`, `start` ("Take notes") and `dismiss`.

The Wayland problem. GNOME's compositor ignores where a window asks to be and won't keep it above other apps. So `widget/controller.ts#createWidget` picks one of two implementations:

- `InProcessWidget`, on X11, or when `CEREAL_WIDGET_IN_PROCESS=1` as in the e2e test: an ordinary always-on-top frameless window of the app.
- `HostedWidget`, on Wayland: it spawns the same Electron binary as `--cereal-widget-host --ozone-platform=x11`. That second process is an *XWayland* client, so it can place itself and stay on top, while the main window stays native Wayland.
  - **Protocol:** NDJSON over an extra pipe on file descriptor 3. Main sends `state`, `recording`, `levels`, `show` and `hide`; the host replies `ready` and `action`.
  - **Isolation:** the host has its own Chromium profile, saves the dragged position together with the bounds of the display it was on, and quits when the pipe closes.
  - **Sizing:** it zooms its page by the app's display scale divided by its own, so the pill matches the app's size on native-resolution XWayland.
  - **Fallback:** three quick crashes (each within 10 s of starting) switch to the in-process widget.

> **THE FOCUS TRICK**
>
> A click in an XWayland window gives the Wayland main window no activation token, so `focus()` does nothing. For widget actions, `showMain(route, remap=true)` hides and re-shows the main window. GNOME does focus a freshly mapped window.

Google Calendar, without a server

`calendar.ts#CalendarService` implements the OAuth 2.0 flow for desktop apps with **your own** Google OAuth client (type *Desktop app*, with the Calendar API enabled). The client ID goes in Settings; the optional client secret goes in `.env`.

1. **Prepare.** Generate a PKCE verifier and challenge (S256) and a random `state`.
2. **Listen.** Start a one-shot HTTP server on `127.0.0.1:<random port>`; this is the *loopback redirect*. It times out after 5 minutes.
3. **Open the browser** to Google's consent page with `scope = calendar.events.readonly`, `access_type=offline` and `prompt=consent`, so Google returns a refresh token.
4. **Receive the code.** The browser comes back to the loopback with `?code&state`. The server checks `state`, shows "Cereal is connected", and closes.

5. **Exchange.** Trade the code and verifier at `oauth2.googleapis.com/token`. The **refresh token** is sealed (chapter 5); the access token is kept in memory until 60 s before expiry and refreshed on demand. `invalid_grant` disconnects.

`sync()` reads the primary calendar from one day back to 14 days ahead, with `singleEvents`, ordered by start, up to 250 events. `toCalendarEvent` keeps only timed events you haven't declined. It drops room resources from attendees and finds a meeting URL in `conferenceData`, `hangoutLink`, or a Zoom, Meet, Teams or Webex link in the location or description. Events replace the stored window in `calendar_events` and are pushed as `calendar:changed`.

The calendar is used in four places:

- **Home** shows "Coming up".
- **Reminders.** Every 30 s, events starting within `notifyMinutesBefore` (default 1 minute) raise a notification. Clicking it opens or creates the event's note and starts recording.
- **Linking.** `current()` (an event started up to 5 minutes ago and not yet ended) links new meeting notes to the ongoing event: title, attendees as `people`, and keyterms.
- **The agent** reads events through two calendar tools.

Attachments

`attachments/service.ts#AttachmentService` processes files locally, the job Granola's `process-attachments` server endpoint does:

1. `chat:attach` **receives the bytes.** `classifyFile` decides the kind (image, PDF, text or office) from the name and type. `verifyBytes` sniffs the content so a renamed file is rejected. Limits: 30 MB per file and per thread, 100 PDF pages per thread. Legacy `.doc` and `.xls` are refused with a hint to save as `.docx` or `.xlsx`.
2. **Store.** Write `<userData>/attachments/<id>/original`, record a `processing` row with its sha256, and return at once.
3. **Process in the background.** Extract text (unpdf for PDFs, mammoth for Word, a small JSZip reader for Excel) into `text.md`, count pages, set the status to `ready` or `failed`, and push `chat:attachment`. The Composer won't send until every chip is ready.
4. **On send,** drafts are bound to the user message. Providers then read them lazily through the run's `FileAccess` and upload them to their own Files API on first use (chapter 9).
5. **Clean up.** Removing a file also deletes the providers' remote copies. Drafts never sent are purged after 24 hours, and deleting a thread removes its files.

Thumbnails and the preview lightbox load from a custom privileged scheme, `cereal-attachment://file/<id>`. The handler validates the UUID, serves text files as `text/plain` with `nosniff` (never rendered as HTML), and is reachable only from Cereal's pages, which the CSP and the sender checks ensure.

CHAPTER 15

Building, testing and shipping

How the code becomes a running app, how it is tested without real APIs, and how the Flatpak keeps it sandboxed.

Everyday commands

Command	What it does
`npm run dev`	electron-vite in dev mode. The renderer is served by Vite with hot reload (`ELECTRON_RENDERER_URL`), and main and preload rebuild on change. Uses its own profile in `~/.config/Cereal`.
`npm start`	Builds the production bundle and runs it unpackaged.
`npm run typecheck`	`tsc --noEmit` for the node side (main, preload, shared) and the web side (renderer, shared) separately.
`npm test`	vitest, run *under Electron's Node* (`ELECTRON_RUN_AS_NODE=1 electron vitest.mjs run`), because better-sqlite3 is compiled for Electron's ABI.
`npm run e2e`	Builds, then drives the real app with Playwright (`tests/e2e/smoke.mjs`).
`npm run flatpak`	Typecheck and tests, an offline Flatpak build, a bundle in `dist/`, and a lint. `-- --install` installs it; `npm run update-app` does the same.

The test pyramid

- **Pure logic** lives in `src/shared` and in pure helpers in main (`settings.pure.ts`, `schedule.ts`, `segmenter.ts`, `placement.ts`). It is tested directly:
  - `shared.test.ts`: the transcript merge, markdown and attribution;
  - `enhanced-notes.test.ts`, `chatbar.test.ts`, `bars.test.ts`, `widget.test.ts`;
  - `agent-fs.test.ts`: path escapes, cron, daylight saving, skill validation.
- **Services with fakes:**
  - `store.test.ts` runs against a real in-memory SQLite.
  - `services.test.ts` runs the Recorder with fake capture and engine, the detector state machine, enhance variables and calendar mapping.
  - `transcription.test.ts` covers the Deepgram URL and a mock WebSocket server, the segmenter and the stabiliser.
  - `attachments.test.ts` covers the file pipeline.
- **The agent.** `tests/agent-harness.ts` builds the whole agent (backend, chat, Assistant) on a temporary database, with `InProcessHost` and a *scripted* `LlmProvider`. Each `ScriptStep` says what the "model" does on one step: stream text or reasoning, call tools, or wait on a promise to test stopping mid-step.
  - `agent.test.ts` covers runs, citations, approvals, resumption after restart, tasks, the inbox and routines.
  - `agent-llm.test.ts` checks each provider's wire translation against mock servers: Anthropic SSE with thinking-signature replay, OpenAI Responses, Gemini, Ollama tool calls, and each Files API.
- **End to end.** The e2e script:
  - starts a fake Deepgram WebSocket server that answers each source with a scripted conversation, and a fake Ollama HTTP server whose "model" emits tool calls (`searchMeetings`, then `draftEmail`);
  - launches the built app with an isolated `CEREAL_USER_DATA` and `CEREAL_WIDGET_IN_PROCESS=1`;

- walks the core loop: new note → type notes → transcribe (real PipeWire capture) → stop → enhance → edit an AI line and explain it → chat with a citation → edit and resend.

GOTCHA

The single-instance lock is taken in `~/.config/Cereal` *before* `CEREAL_USER_DATA` applies. If an unpackaged Cereal (`npm run dev`) is running, the e2e app quits instantly with exit code 0. Run e2e with `XDG_CONFIG_HOME=<scratch dir>`, as `scripts/flatpak.sh --e2e` does.

Environment switches

Variable	Effect
`CEREAL_USER_DATA`	Use another profile folder (database, `.env`, models, agent folder).
`CEREAL_DEEPGRAM_URL`, `CEREAL_DEEPGRAM_LISTEN_URL`	Point streaming and prerecorded Deepgram at a test server.
`CEREAL_ANTHROPIC_URL`, `_OPENAI_URL`, `_GEMINI_URL`	Point the providers at mocks.
`CEREAL_AGENT_IN_PROCESS=1`	Run the agent loop in main instead of the utility process.
`CEREAL_WIDGET_IN_PROCESS=1`, `CEREAL_WIDGET_DEBUG_PORT`	Keep the widget in the app; open DevTools on the XWayland host.
`DEEPGRAM_API_KEY` and the other keys	Used when `.env` leaves them empty.

The Flatpak

`flatpak/dev.cereal.app.yml` builds on the Freedesktop 26.08 runtime with the **Electron BaseApp**, which provides zypak, the sandbox shim that replaces Chromium's setuid sandbox inside Flatpak. It has two modules.

1. **whisper-cpp v1.9.5**, built with CMake. All CPU variants are compiled and loaded dynamically from `/app/lib/ggml`, so one bundle runs at full speed on any x86-64 CPU. Only `whisper-server` is kept in `/app/bin`.
2. **cereal.**
    1. `npm ci --offline` from `generated-sources.json`, every npm tarball and Electron's zip pinned by hash, so the build has no network access.
    2. `electron-vite build`.
    3. `electron-builder --dir`, using the pre-fetched Electron.
    4. Copy the result to `/app/cereal` and drop `chrome-sandbox`.
    5. `patch-electron-desktop-filename`, so the Wayland app id matches the desktop file.
    6. Install the launcher, desktop file, metainfo and icons.

The launcher, `flatpak/cereal.sh`, sets `TMPDIR` to the app's runtime folder, so a second launch finds the first instance's singleton socket, and then `exec zypak-wrapper /app/cereal/cereal`.

Sandbox permissions

Granted	Why
`--share=network`	The AI and transcription APIs, Ollama, the Google loopback.
`--socket=wayland`, `--socket=x11`, `--share=ipc`	The main window on Wayland; the widget as an XWayland window.
`--socket=pulseaudio`, `--filesystem=xdg-run/pipewire-0`	UI sounds and dictation; `pw-record` and `pw-dump`.

Granted	Why
`--device=dri`, notifications, StatusNotifierWatcher	GPU, reminders, the tray.

Deliberately not granted:

- home or host files: the portals cover attachments and "Open .env";
- `org.freedesktop.secrets`: the Secret portal protects only Cereal's own token;
- `org.freedesktop.Flatpak`, because `flatpak-spawn --host` would escape the sandbox.

Data lives in `~/.var/app/dev.cereal.app/config/Cereal/`: `cereal.db`, `.env`, `models/`, `attachments/`, `agent/` and `widget-host/`.

The security model at a glance

Threat	Mitigation
Injected content in the UI reaching the system	Sandboxed, context-isolated renderers; a three-member bridge; channel allowlists; a sender URL check; a strict CSP; no navigation or `window.open`.
Prompt injection through meeting content making the agent act	No shell. File tools confined to one folder (lexical and realpath checks). Drafts are never sent. Writes outside `memory/` and new routines need approval.
Other apps reading API keys	Keys in the Flatpak's private folder with mode 0600 (reset on read); never sent to the renderer.
Google token theft from disk	Async `safeStorage` through the Secret portal; a `v10` fallback key counts as no protection and is reported.
A malicious attachment	Type sniffing, size limits, text served as `text/plain` with `nosniff`, the attachment scheme reachable only from Cereal's pages, and the run checks a file belongs to its thread.
Not covered	Programs you run outside Flatpak can read your files. The database is not encrypted, unlike Granola's SQLCipher; use full-disk encryption.

CHAPTER 16

One meeting, end to end

Every subsystem in Part I, in the order they fire, for a 30-minute customer call with Google Calendar connected, Deepgram for transcription and Claude for AI.

When	What happens	Where in the code
09:00	Calendar sync (every 5 minutes) has the 10:00 "Acme pilot" event in `calendar_events`.	`CalendarService.sync` → `replaceCalendarEvents` → `calendar:changed` → Home "Coming up".
09:59	A reminder notification: "Acme pilot is starting".	`checkUpcomingMeetings` (30 s timer), `notifyMinutesBefore = 1`.
10:00	You click it. The event's note opens and recording starts.	`noteForEvent` creates `documents` with `calendar_event` and `people`; `showMain('/note/…')`; `recorder.start(id)`.
10:00	Two Deepgram sockets open, two `pw-record`s start.	`createEngine` reads `DEEPGRAM_API_KEY` from `.env`; keyterms are the dictionary, your name and the attendees; `recording:state` = recording.
during	Words appear in the transcript, growing in place.	Deepgram interims (key `w7`) map to one chunk id; `transcript:chunks` → `mergeTranscriptChunks`. Finals are upserted to SQLite.
during	You type "Pilot budget $50k". It saves 500 ms after you stop typing.	`Notepad` → JSON, plain text and Markdown → `documents:update` → FTS reindex.
during	You switch to Zoom; the floating pill appears.	Main window `blur` → `updateWidget` → `HostedWidget.show` (XWayland), with levels streamed over fd 3.
10:30	You leave the call. 20 s later Cereal stops: "Meeting ended".	`MeetingDetector` sees no call app in `pw-dump` for 20 s → `stopRecording` → `CloseStream`, last finals, interims finalised, `finalizeTranscript`.
10:31	You click *Enhance notes*; text streams into a new tab.	`panels:enhance` → `AiService.enhance`: variables (template Customer call, 4 headings) → Claude Opus streaming → `panel:updated` every 150 ms.
10:31	Your line stays black; the AI's lines turn grey.	`computeGeneratedLines` → `generated_lines`; the `GeneratedLines` plugin. `autoTitle` skips this note: it already has the calendar event's title.
10:32	You hover "Legal must approve by Friday" → Transcript Summary with two quotes.	`transcript-summary:start` → fast model → `matchQuote` → click opens the transcript at the chunk.
10:33	"Draft a follow-up email" in the chat bar.	`chat:send` → `ChatAgent.send` → run (`note-chat`) → worker `claim` → `step` → `draftEmail` → email card. `[1]` cites the note.
10:34	"Add that to my notes" → a line appears in My notes, with Undo.	`editMeetingNotes` checked in main → applied in the editor (`applyNoteEdit`) → `chat:tool-applied`.
next day 09:00	Your routine fires; the Assistant summarises yesterday's meetings and updates Home.	`tick` → due routine → inbox (scheduler) → `drain` → conversation run → `listMeetings` + `readMeetings` → `publishHome` → notification.

EXERCISE

Pick any row and follow it in the code. Start from the IPC channel or timer named in the right column, go to its `handle(…)` in `app.ts`, then into the service. With chapters 3 and 10 in mind, you can trace any feature in a few minutes.

PART II

Granola

What a reverse-engineering pass over Granola's desktop app shows about the parts Cereal doesn't have: accounts, sync, sharing, Google connections, integrations, workspaces and updates. It also recaps how Granola runs the features Cereal copied.

CHAPTER 17

How Granola was studied, and its overall shape

Granola is a thin, careful client over a large backend. Almost everything Cereal lacks lives on that backend. The client only shows us the backend's interface.

Method

- **Artifacts.** The Windows installers for Granola **7.663.2** (production) and **7.668.0** (beta), verified against the updater's sha512 and extracted to `~/re/granola/<version>/asar`. The macOS download is a 750 KB stub installer that asks `api.granola.ai/v1/get-versions` which build to fetch.
- **REA.** `analyze_javascript_application` was run over the extracted application. This pass covered 7.668.0, with the main bundle, the preload and 11 renderer chunks selected (sync, API, entity schemas, cache, URL map). Parent Evidence: `ev_c3cea9ae6db7…`; summary `ev_a11cc836f42d…`. It recorded 3,862 findings over 12 parsed files, with partial semantic coverage. The session is exported to `~/re/granola/evidence/granola-7.668.0-platform-evidence.json`. Earlier passes, in `docs/granola-*.md`, cover the agents, UI and chat bar.
- **Reading bytes.** Granola's API calls go through dynamic wrappers like `$(token, "endpoint-name", {input})`, so static graph tracing can't follow request fields. Exact shapes came from reading the located regions as *inert text*: zod schemas, enums, endpoint maps, the SQL schema strings and handler code. **Nothing was executed**, and the Windows build cannot run on this host. Server behaviour is therefore inferred from what the client sends and expects back.

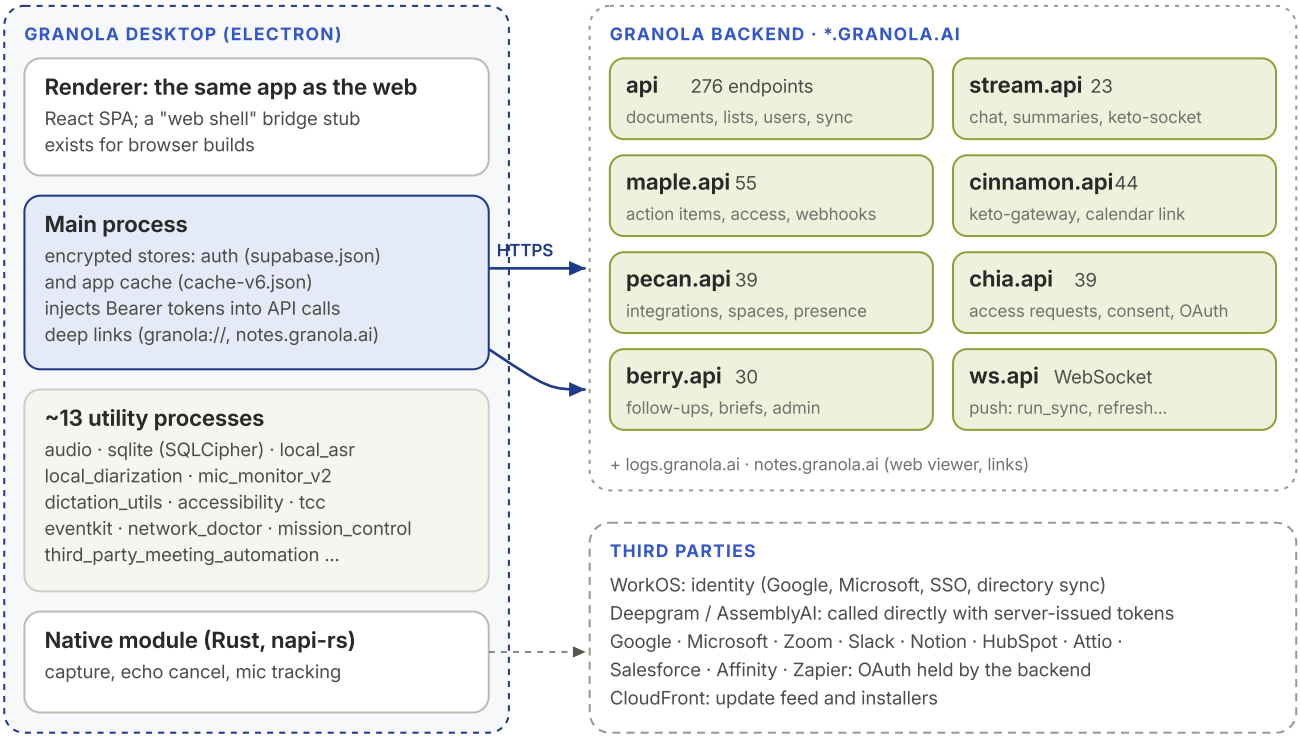


Figure 17.1 Granola's shape. Endpoint counts are names in the client's URL map (`types-*.js`) O : about 545 entries across seven API hosts. The host-to-service mapping beyond the names is inferred I .

Things worth knowing about the client

- **One URL map.** A single object maps every endpoint name to a full URL, for example `"calendar-link-auth": "https://pecan.api.granola.ai/v1/calendar-link-auth"` O . Callers use names, so moving a service between hosts is a configuration change.

- **The request wrapper** `$(token, name, options)` takes `input`, `retries`, `method`, `signal`, `additionalHeaders`, a `sensitivePayload` flag that keeps bodies out of logs, and a `queue` (`documents` or `document-panels`) that serialises writes per resource type O. Two headers recur: `X-Granola-Workspace-Id` for workspace-scoped calls and `X-Granola-Time-Zone`.
- **Streaming endpoints** (`stream.api`) return delimiter-separated JSON chunks or SSE frames, read incrementally O.
- **The web shell.** The renderer bundle contains a web-shell bridge: `localStorage` session storage, a `BroadcastChannel`, and `fetch('/api/session')` stubs for desktop-only features O. The same React app very likely also runs as Granola's web app I.
- **Fuses.** `RunAsNode` and `--inspect` are disabled, and ASAR integrity is enforced O.

Endpoints by domain

Domain	Representative endpoint names O
Identity	`workos-auth-complete`, `refresh-access-token`, `auth-soft-check`, `auth-handoff-complete`, `login-check`, `revoke-session`, `list-user-sessions`, `get-user-info`
Documents and sync	`get-documents`, `get-documents-batch(-stream)`, `get-documents-delta`, `get-document-set(-stream)`, `create-document`, `update-document`, `upsert-document`, `hard-delete-document`, `get-ydoc`, `insert-collaborative-documents`, `sync-push`, `update-document-presence`
Panels and transcripts	`create-document-panel`, `update-document-panel`, `get-document-panels`, `insert-transcriptions`, `get-document-transcript`, `delete-transcription-chunks`, `get-deepgram-token`, `transcribe-audio`
Sharing and access	`add-users-to-document`, `get-users-with-access`, `check-document-access`, `request-document-access`, `grant-document-access-request`, `update-documents-sharing-link-visibility`, `get-shared-documents`, `get-note-opengraph-data`, `get-public-document-panel`, `send-notes-notification`
Folders ("document lists")	`create-document-list(-v2)`, `add-document-to-list`, `add-users-to-document-list(-v2)`, `get-document-list-opengraph-data`, `grant-folder-access-request`
Calendar	`calendar-link-auth`, `calendar-link-auth-complete`, `get-google-events`, `refresh-google-events`, `refresh-calendar-events`, `get-selected-calendars`, `google-calendar-webhook`, `outlook-calendar-webhook`, `create-calendar-event`, `find-free-calendar-slots`
Integrations	`list-available-integrations`, `integration-connection`, `integration-oauth-callback`, `slack-oauth-callback`, `notion-oauth-callback`, `hubspot-oauth-callback`, `save-to-integration`, `post-slack-message`, `save-to-notion`, `add-zapier-connection`, `create-webhook-endpoint`, `create-public-api-key`
Workspaces	`create-workspace`, `join-workspace(-via-code)`, `create-workspace-invite-link`, `set-workspace-roles`, `update-workspace-sso-config`, `generate-sso-portal-link`, `generate-directory-sync-portal-link`, `move-documents-to-workspace`
AI	`generate-summary`, `chat-with-documents`, `llm-proxy-stream`, `view-source-summary-web`, `keto-gateway`, `keto-socket`, `raisin`, `search-meetings-turbopuffer`, `generate-document-embeddings`
Billing and growth	`create-subscription`, `get-paywall-status`, `create-referral-link`, `create-tracked-link`, `validate-promo-code`, `stripe-event-handler`
Updates	`check-for-update`, `get-versions`, `get-release-channels`, `update-release-channel`, `download-latest`

CHAPTER 18

Identity: sign-in, tokens and deep links

Granola has accounts; Cereal doesn't. Granola's sign-in is a browser round trip that ends in a custom-scheme deep link, after which the main process quietly attaches a token to every API call.

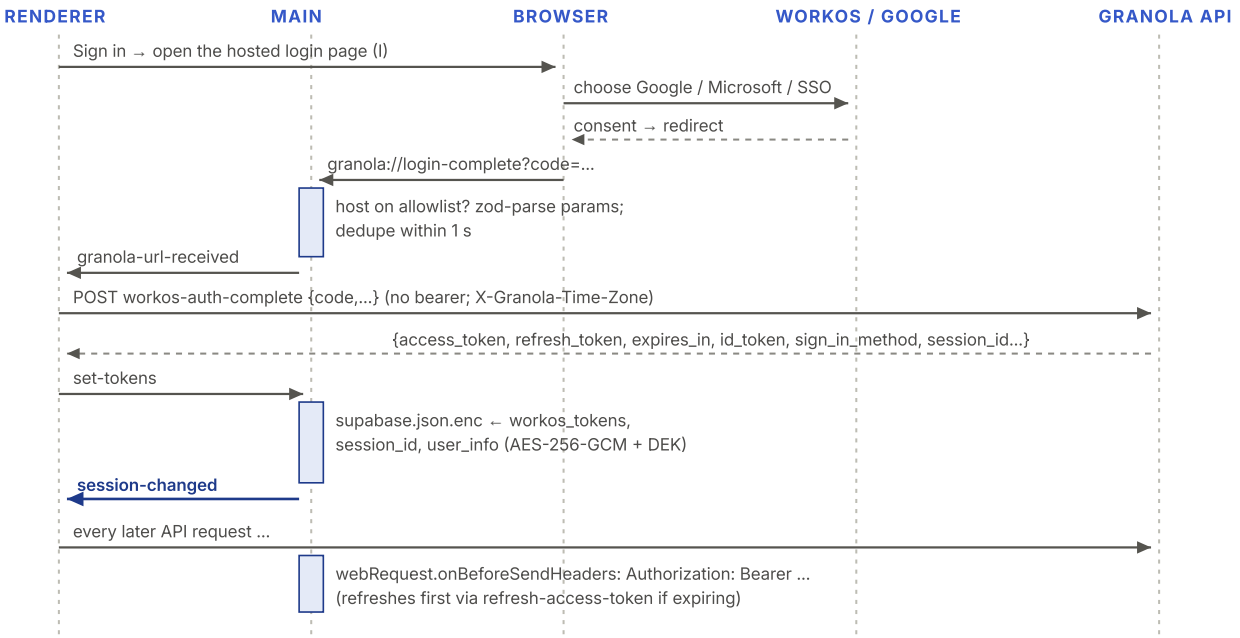


Figure 18.1 Granola's sign-in. Steps are O except the first: the exact login URL is built dynamically I.

The pieces

- **Identity provider: WorkOS.** The logout fallback calls `api.workos.com/user_management/sessions/logout`. The token record is `workos_tokens` with a `sign_in_method` such as `GoogleOAuth`, and user records carry `workos_id`, `google_sub`, `microsoft_sub`, `zoom_id` and `has_logged_in_with_sso` O. "Sign in with Google" therefore federates *through* WorkOS. The desktop app never handles Google's own sign-in tokens I.
- **The deep-link scheme.** Packaged builds register `granola`, development builds `granola-dev`, with `app.setAsDefaultProtocolClient`. URLs arrive through `open-url` on macOS, `continue-activity` for universal links, and command-line arguments on Windows O.
- **The URL router** accepts only 15 hosts: `login-complete`, `join-list`, `new-document`, `put-document`, `try-recipe`, `open-workspace`, `open-document`, `open-settings`, `set-feature-flags`, `turbopuffer-search`, `zoom-redirect`, `start-demo`, `try-voice-typing`, `ws-migration-banner` and `open`. Each host's query string is validated with its own zod schema O. `login-complete` takes:
  - `code` (required);
  - `sso`, `ssoCode`, `handoff`, `needsOAuthLinking`;
  - `inAppCalendarLinking` and `calendarLinkingFlow` (chapter 21);
  - marketing attribution ids (`gclid`, `fbclid`, `dub_id` and others).

  Identical events within a second are dropped.
- **Web links open the app too.** `https://notes.granola.ai/d/<id>` becomes `open-document`, `/f/<id>` becomes `join-list` (a folder invite), and `/t/<slug>` becomes a tracked link O. Chapter 20 covers them.

Token storage and use

- **Where.** Tokens live in main's encrypted store, `supabase.json`: an AES-256-GCM file keyed by a `safeStorage`-wrapped data key (chapter 5). The keys are `workos_tokens`, `session_id` and `user_info` O . The file name and a matching `supabase-storage-v1` key in the web shell are legacy names, from before WorkOS I .
- **Attaching them: the clever bit.** Main registers `session.webRequest.onBeforeSendHeaders` for *every URL in the endpoint map except* `refresh-access-token`, and adds `Authorization: Bearer <access_token>`, refreshing first if the token is near expiry O . Renderer code can call the API without managing tokens, and refresh logic exists in exactly one place.
- **Several accounts.** `get-stored-accounts` and `switch-to-account` swap the stored tokens and clear the cache when the user id changes O .
- **Logout** calls `revoke-session` with `intent: logout`. `list-user-sessions` exists for session management O .

NOT IN CEREAL

Cereal has no accounts, sessions, deep-link scheme or account switching. It stores one Google refresh token, obtained by its own OAuth flow (chapter 14). Part III discusses whether it needs any of this.

CHAPTER 19

The sync engine

Granola keeps a full local copy of your data, so it works offline and opens fast. The server is the source of truth, and a WebSocket tells the client when to pull. Cereal's whole storage chapter is the "local copy" half of this.

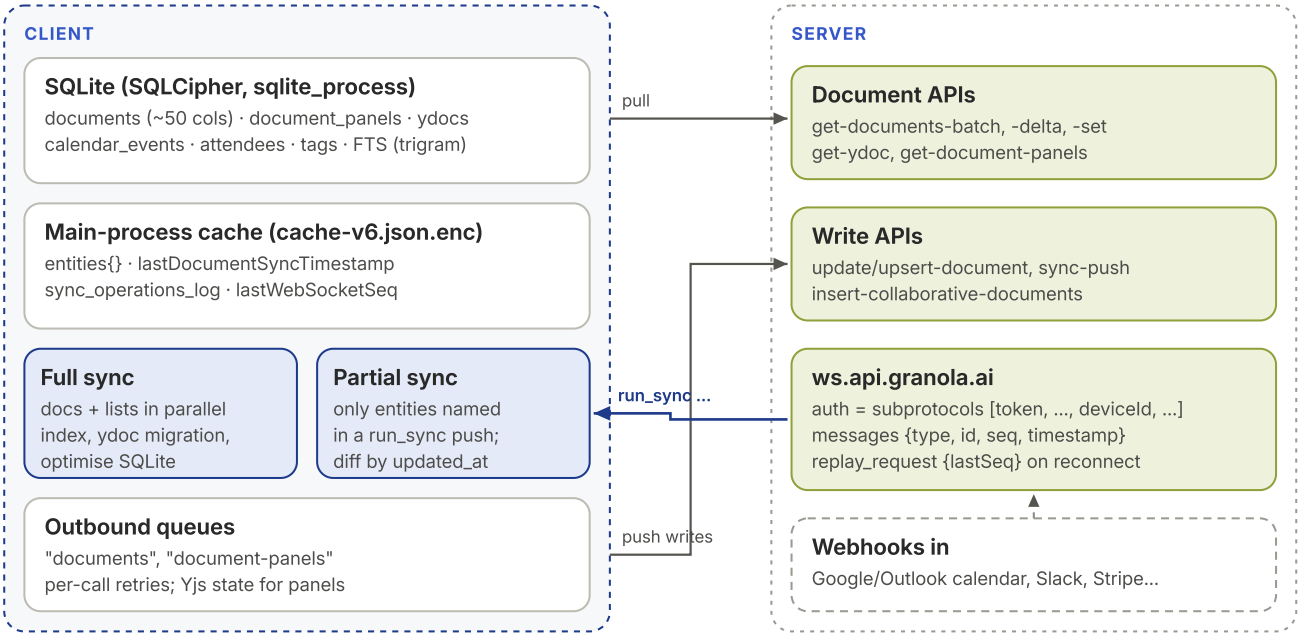


Figure 19.1 Granola's sync. A push names what changed; the client pulls it and diffs it into local state. Everything shown is `O` in the client; server internals are inferred.

Local state on two levels

- **SQLite** holds documents, panels, Yjs states (`ydocs`), calendar events, attendees, tags, pre-meeting briefs and dictation history. Its `documents` table carries server-sync columns: `user_id`, `workspace_id`, `visibility`, `sharing_link_visibility`, `is_shared`, `is_shared_direct`, `document_user_role`, `ydoc_state`, `ydoc_version`, `ydoc_cached_at`, plus generated columns `event_id` and `display_time` for fast lists `O`.
- **An application cache** lives in main as one JSON state at version 9, written encrypted and throttled. Renderers update it by dispatching actions (`cache:dispatch`). It holds normalised `entities` by type, sync bookkeeping (`lastDocumentSyncTimestamp`, `hasCompletedFullDocumentSync`, `lastGoogleCalendarSyncTimestamp`, `backfilledYdocVersion`), the outbound `sync_operations_log`, pending view and audit events, and WebSocket position (`lastWebSocketSeq`, the last 100 `processedWebSocketMessageIds`). A `prune` action drops completed operations older than a day and audit events older than a week `O`.

Pulling

- **Full sync.** Guarded by an in-progress flag. It syncs documents and document lists in parallel, marks the full sync complete, starts search indexing on the first run, migrates panels to Yjs, cleans up orphaned Yjs states, and optimises SQLite `O`.
- **Partial sync.** Triggered by a push naming entity ids. Documents are fetched in batches, then split into the user's own documents and documents *shared directly* with them, and stored separately. A requested id that no longer comes back means access was revoked: the local copy is deleted and logged as `document-removed-access-revoked` `O`. Lists, MCP connections, chat threads and messages sync the same way.

- **Generic entity sync.** Each entity type has a config with flags such as `localOnly` and a `featureFlag` gate. The client fetches the remote set and diffs it with local by `updated_at`, producing add, update and delete operations applied as one `applyOperationsBatch`. It skips anything a pending local operation is about to change, so a sync never overwrites an unsent edit O .

The push channel

A reconnecting WebSocket to `wss://ws.api.granola.ai` O :

- **Authentication** goes in the *subprotocol list*: `["token", <fresh access token>, "deviceId", <id>]`. Browsers can't set headers on WebSockets, so this is the standard workaround.
- **Resume.** On open, the client sends `{type: "replay_request", lastSeq}`, so the server can resend what it missed. Messages carry an `id` (deduplicated) and a `seq` (a high-water mark).
- **Keepalive.** The client sends `ping` on an interval with jitter. A late `pong` marks the connection in error. Reconnect delays are randomised and grow exponentially, and the connection is re-checked when the network comes back.
- **Message types:**
  - sync and calendar: `run_sync` (entities, an immediate flag), `refresh_google_events`, `sync_user_preferences`;
  - integrations and workspaces: `refresh_slack_integration`, `sync_user_workspaces_access`, `workspace_offer_refresh`, `refresh_workspace_automations`, `refresh_automation_matches`, `folder_integration_check_complete`, `integration_save_status`;
  - meeting state: `request_transcription_state`, `meeting_consent_status_updated`, `zoom_rtms_consent_update`;
  - AI results: `refresh_pre_meeting_briefs`, `refresh_follow_up_emails`, `meeting_classification_complete`, `dispatch_classification_complete`;
  - plus `pong`.

KEY IDEA: HINTS, NOT DATA

The push channel carries *hints*: "these entities changed", "refresh calendar". The client then pulls through ordinary HTTP endpoints. This is the same "push a hint, pull the data" rule Cereal uses between main and renderer (chapter 3), at internet scale. It makes the channel cheap and idempotent: a duplicated or replayed hint just causes one redundant pull.

Pushing changes, and collaboration

- **Writes** go through named per-resource queues with retries. Panel writes get up to 5 retries on create and 1 on update. Transcripts are uploaded with `insert-transcriptions` O .
- **Collaborative content is Yjs.** Panel bodies ("summary" content) and private notes are kept as Yjs document states. The `ydocs` table stores state per content id and version. A migration converts older ProseMirror JSON panels into Yjs states and uploads them with `insert-collaborative-documents`, accepting the server's state when another client won the race. Conflict modes like `clientWinsDisjoint` appear O .
- **Presence** endpoints (`update-document-presence`, `get-meeting-presence`, `report-meeting-presence`) suggest who-is-here indicators I .

NOT IN CEREAL

Cereal has one device and no server, so it has no sync, conflict handling or collaboration. Its notes are plain TipTap JSON, not Yjs. That is the right call for a local app, and the biggest structural difference if multi-device or sharing ever comes.

CHAPTER 20

Sharing and shareable links

A Granola note can be private, shared with named people, shared with a workspace, or reachable by anyone holding a link. Four fields on the document and a handful of endpoints carry all of it.

The data model

Field (on `documents`)	Values and meaning O
`visibility`	`private` · `workspace` · `public`: who can find the note inside Granola.
`sharing_link_visibility`	`private` ("Members only") · `organisation` ("Everyone @company") · `public` ("Anyone with the link"): what the note's link grants.
`has_shareable_link`, `public`	Whether a link exists and is live.
`is_shared`, `is_shared_direct`, `has_direct_shares`	Shared at all; shared with *me* by someone else; shared by me with named people.
`document_user_role`	`OWNER` · `COLLABORATOR` · `VIEWER`: my role on this note.
`workspace_id`, `integration_shares`	Which workspace owns it; where it was posted (Slack, Notion, Attio, Zapier, HubSpot, Affinity).

Folders ("document lists") have the same `sharing_link_visibility` triple and their own member lists. Workspaces set a default `sharing_link_visibility` and a `link_sharing_allowed_domains` list; one toast reads "Sharing is now limited to your workspace domain". Spaces are typed `workspace`, `crm` or `private` O .

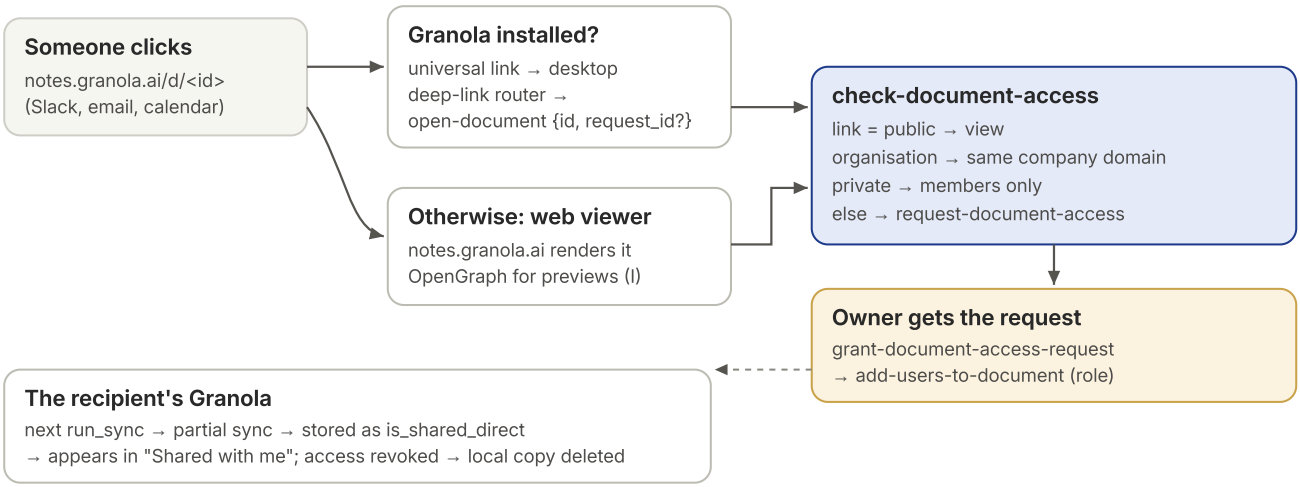


Figure 20.1 How a shared link resolves. The client-side steps are O ; the server's access decisions are inferred from the endpoints and enums I .

Link formats

URL	What it is O
`https://notes.granola.ai/d/<id>`	A note. The desktop app opens it as `open-document`; an optional `?request_id=` links to an access request.
`https://notes.granola.ai/f/<id>`	A folder invite (`join-list`).
`https://notes.granola.ai/t/<slug>`	A tracked link (`create-tracked-link`, `retrieve-tracked-link`) for referrals and attribution.

URL	What it is O
`granola://open-document?document_id=…`	The in-app form, used by the app's own popups too.

Copy-to-clipboard code recognises these links in pasted Markdown, so a shared Granola link pasted into a note stays a link to the note O .

The share UI

- **The toolbar.** A split pill: a "Shared — manage access" or "Private — manage access" button showing collaborator avatars, and a "Copy link" button with its own keyboard shortcut. The group is labelled "Share and copy link" O .
- **The dialog.** Invite people and set their role. A "Link access" picker offers *Members only*, *Everyone @domain* and *Anyone with the link*; only the options the workspace allows are listed. Toasts confirm "Sharing links switched on / off / to organization only" O .
- **Related actions:** share with attendees (`send-notes-notification` emails them), post to Slack, save to Notion or a CRM, move notes to a workspace, and transfer ownership (`request-transfer-notes`, `transfer-notes`) O .

NOT IN CEREAL

Cereal's share menu copies Markdown or the transcript to the clipboard. Nothing comparable to a link exists, because there is nowhere to host the note and no identity to check access against. Chapter 25 sketches options.

CHAPTER 21

Google connections

"Connect Google" means three different things in Granola: signing in, the calendar, and Gmail plus Google Docs for the agent. All three share one design decision: Google tokens live on Granola's servers, not in the app.

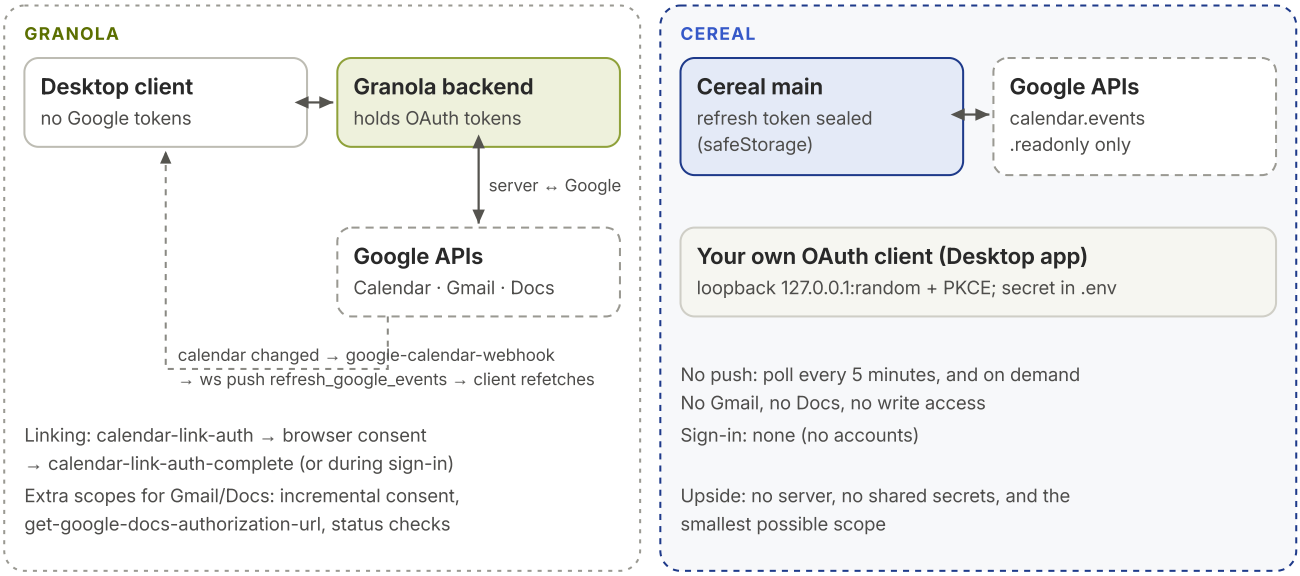


Figure 21.1 Google connections compared. A search of all of Granola's shipped JavaScript finds no Google API endpoint, only a public-suffix list containing `googleapis.com` O . Every Google call is therefore made by Granola's servers I .

1. Signing in with Google

Sign-in is a WorkOS federation (chapter 18). The token record's `sign_in_method` is, for example, `GoogleOAuth`, and the user record keeps `google_sub` and `has_active_oauth_connection` O . Microsoft (`microsoft_sub`), Zoom (`zoom_id`, `zoom-login-complete`) and enterprise SSO (`has_logged_in_with_sso`, `generate-sso-portal-link`) are parallel paths.

2. The calendar

- **Linking.** Either during sign-in, where `login-complete` carries `inAppCalendarLinking` and `calendarLinkingFlow`, or later: `calendar-link-auth` on `pecan.api` returns a consent URL for the browser, and `calendar-link-auth-complete` on `cinnamon.api` finishes it O . Google grants the token to Granola's server I .
- **Reading.** The client asks Granola's backend: `refresh-calendar-events`, `get-google-events`, `refresh-google-events`, `get-calendar-event`. Events are cached in local SQLite (`calendar_events`), and `lastGoogleCalendarSyncTimestamp` sits in the cache O .
- **Freshness.** Three triggers O :
  1. a polling interval set by a feature flag;
  2. the window becoming visible or the Home view opening, skipped if the last sync is recent;
  3. **push**: Google's change notifications reach `google-calendar-webhook` on the server I , which sends `refresh_google_events` over the WebSocket, and the client refetches after a debounce.
- **Choice and writing.** `get-selected-calendars`, `set-selected-calendars` and `reset-selected-calendars` choose which calendars count. The server also writes: `create-calendar-event`, `execute-calendar-action`, `find-free-calendar-slots`, `query-calendar-availability`, and agent tools such as `editCalendarEvent`, `respondToCalendarEvent` and `createFocusTime` O .

- **Outlook** has the same shape (`outlook-calendar-webhook`, Microsoft consent endpoints).

3. Gmail and Google Docs for the agent

- **Incremental consent.** These scopes are requested separately, when first needed. `get-google-workflow-authorization-status`, `get-google-docs-authorization-url` and `-status`, and `modify-google-integrations {action: 'get' | …}` manage them `O` .
- **Mid-conversation.** The chat agent has `resolveConnectorAccess {connector: gmail | slack | google_docs}` and `requestGoogleWorkflowAuthorization`, so it can ask for access in the middle of a chat `O` .
- **Tools.** Then `searchEmailThreads*`, `readEmailThread`, `getEmailWritingStyle`, `searchGoogleDocs`, `readGoogleDoc`, and a cross-source `search` over Slack, Granola, Gmail and Calendar. All run server-side.

THE TRADE-OFF

Server-held tokens give Granola calendar push, background work while your laptop sleeps, the same connection on web and mobile, and one consent screen with a verified OAuth app. The cost is that Granola can read your calendar and mail. Cereal's design is the opposite: your own OAuth client, a read-only scope, the token on your machine, and polling instead of push. Part III considers how far Cereal could go without giving that up.

CHAPTER 22

Integrations, workspaces and updates

The rest of Granola's server-backed surface, in less depth.

Integrations

The providers seen are Slack, Notion, HubSpot, Attio, Affinity, Salesforce, Zapier, Linear (as Dispatch drafts) and Zoom, plus generic webhooks, a public API with keys, and MCP servers O. Connecting one is a server-side OAuth round trip that the client *polls* O:

1. Ask the backend for the provider's status: `{isConnected, canIntegrate, authUrl}`.
2. Open `authUrl` in the browser, or a prepared popup in the web app.
3. The provider redirects to *Granola's server* (`slack-oauth-callback`, `notion-oauth-callback`, `integration-oauth-callback`…), which stores the token.
4. Meanwhile the client polls the status, quickly at first and then more slowly, waking immediately when the window regains focus, until it is connected or a timeout passes. Then it refreshes the integration list.

Using integrations is also server-side:

- `save-to-integration` and `search-integration(-associations)`, for example attaching a note to a CRM deal;
- `post-slack-message` and `list-slack-channels`;
- folder automations: `link-folder-webhook`, `get-folder-zapier-integrations`, and the `refresh_automation_matches` push;
- per-workspace connector admin controls.

Progress comes back as `integration_save_status` pushes O.

Workspaces and teams

- **Lifecycle:** `create-workspace`, `join-workspace` and `join-workspace-via-code`, invite links and emails, seat requests, roles (`set-workspace-roles`), member statuses, `leave-workspace`, `delete-workspace` O.
- **Enterprise:** SSO configuration and a self-serve portal link (`generate-sso-portal-link`), and directory sync (`generate-directory-sync-portal-link`), consistent with WorkOS's admin portal I.
- **Policy fields on a workspace:** `privacy_mode_enabled`, `transcript_retention_hours`, `sharing_link_visibility`, `allow_moving_notes`, `pre_call_email_enabled`, affirmative consent, plus user groups and spaces O.
- **Scoping.** Requests carry `X-Granola-Workspace-Id`. The client keeps an `activeWorkspaceId` and refetches integrations when it changes.

Auto-update

- **Mechanism:** `electron-updater` with its feed URL set to the `check-for-update` endpoint, which redirects to `https://dr2v7l5emb758.cloudfront.net/<version>/latest.yml` O.
- **Settings:** `autoDownload` only when packaged; downgrades disallowed; differential downloads off unless a flag turns them on. Update requests carry the user's Bearer token plus `X-Granola-Platform` and `X-Granola-Device-Id` O.
- **Managed devices:** an administrator can switch auto-update off through managed preferences: `/Library/Managed Preferences/com.granola.app.plist` on macOS, a registry policy on Windows O.
- **Channels:** production and beta (`get-release-channels`, `update-release-channel`, `get-versions`). The macOS stub installer asks `get-versions` which build to download O.

NOT IN CEREAL

There are no integrations beyond Google Calendar, no workspaces and no auto-update. Cereal is reinstalled from a new `.flatpak` bundle. Publishing to a Flatpak remote (Flathub, or a self-hosted repository) would give updates for free through `flatpak update`.

CHAPTER 23

The copied features, Granola's way

A recap of how Granola runs what Cereal reimplemented locally, with pointers to the detailed specs in `docs/`.

Feature	Granola O	Cereal
Audio capture	Native Rust module in `audio_process`; two PCM16 streams; echo cancellation; Windows render-loopback that follows the meeting app's device.	`pw-record` × 2 with the sink monitor; no echo cancellation.
Live transcription	Deepgram nova-3 with backend-issued short-lived tokens, a fallback host, and AssemblyAI as an alternative; local Parakeet (sherpa-onnx) or whisper-server with Silero VAD.	Deepgram with your key; whisper-server with an energy VAD and the same timings.
Speakers	Local diarization process, voice fingerprints, speaker-name suggestions.	Only "You", "Them" and "Speaker".
Enhance notes	Server prompts by `prompt_slug` and client variables; streamed with Yjs state and `generated_lines`.	Same variables and attribution; own prompts; any provider.
Templates and recipes	Server catalogue, shared across a workspace, with usage tracking and public recipe pages with OpenGraph data.	Local tables; built-ins seeded on start.
Search	Local trigram FTS plus server vector search (`search-meetings-turbopuffer`, embeddings).	Local FTS5 BM25 only.
Ask chat	Server agent, 82 tools, MCP and connectors, web search.	Local agent, 26 tools.
Assistant	Keto cloud workers with bash, plus Hermes, Granny voice, Dispatch and reminders.	Local runtime with sandboxed files; conversation, tasks, routines.
Meeting detection	Native mic-activity tracker; meeting-consent flows; in-meeting chat notices; a virtual webcam watermark.	`pw-dump` polling.
Floating widget	"Nub" window; reminder and notification card windows.	Nub, with an XWayland host on Wayland.
Storage	SQLCipher in a utility process; encrypted cache; server copy.	Plain SQLite in main; single copy.

PART III

The gap

What Cereal lacks compared with Granola, which of those gaps need a server, and how Cereal could close the important ones without giving up being local-first.

CHAPTER 24

Capability by capability

Three questions decide each gap: does Cereal have it, does it need a server, and does it matter for one person on Linux?

Capability	Cereal	Needs a server?	Notes
Two-sided live transcription	✔ Have	no	PipeWire capture; Deepgram or Whisper.
Enhance notes, templates, grey AI lines	✔ Have	no	Same variables and attribution as Granola.
Transcript Summary	✔ Have	no	Quote matching by sliding window instead of Fuse.js.
Agentic chat with citations, files, recipes	✔ Have	no	26 tools against Granola's 82.
Assistant: tasks, routines, memory, skills	✔ Have	no	Local runtime; no shell by design.
Google Calendar (read)	◖ Partial	no	Your own OAuth client; polling; primary calendar only.
Google sign-in	✘ Missing	yes (identity)	Only meaningful once there are accounts.
Gmail and Google Docs for the agent	✘ Missing	no	Possible with incremental scopes on your own client.
Calendar writes	◖ Drafts	no	Today a prefilled Google Calendar page; could be a real write with approval.
Calendar push updates	✘ Missing	**yes**	Google push needs a public HTTPS endpoint; use incremental `syncToken` polling instead.
Shareable links	✘ Missing	**yes** (hosting)	The biggest visible gap. Chapter 25 sketches an end-to-end-encrypted option.
Share with named people, roles	✘ Missing	**yes** (identity + access)	Needs accounts.
Multi-device sync	✘ Missing	**yes** (relay)	Needs a CRDT or sync protocol.
Real-time collaboration	✘ Missing	**yes**	Granola uses Yjs; Cereal stores TipTap JSON.
Slack, Notion, CRM integrations	✘ Missing	no (with tokens)	Internal tokens or webhooks in `.env` work without OAuth; OAuth apps need redirect hosting.
MCP servers for the agent	✘ Missing	no	A local MCP client opens many integrations at once.
Web search for the agent	✘ Missing	no	Provider server tools (Claude `web_search`) or a search API key.
Workspaces, SSO, admin policy	✘ Missing	yes	Not relevant to a single-user app.
Auto-update	✘ Missing	repository only	A Flatpak remote gives `flatpak update` for free.
Echo cancellation	✘ Missing	no	PipeWire `echo-cancel` module.
Speaker diarization	✘ Missing	no	Deepgram `diarize=true` on the system stream; local models.
Database encryption	✘ Missing	no	SQLCipher (`better-sqlite3-multiple-ciphers`) with a key sealed by `safeStorage`.
Semantic search	✘ Missing	no	`sqlite-vec` plus provider or local embeddings.

THE PATTERN

Every gap that needs a server needs it for one of three reasons: **hosting** (someone else must reach the content), **identity** (someone must be told apart from someone else), or **reachability** (a third party must call you, as with webhooks or calendar push). Cereal can add hosting cheaply and stay private. Identity and reachability change what Cereal is.

CHAPTER 25

A roadmap for the missing pieces

Design sketches, not decisions. Each fits Cereal's existing patterns: a service in main, an IPC domain, settings in `.env`, and agent tools with approval policies.

1. Shareable links, end-to-end encrypted

The smallest design that gives "anyone with the link can view" without accounts, and without the host being able to read the notes. Excalidraw and PrivateBin use the same idea.

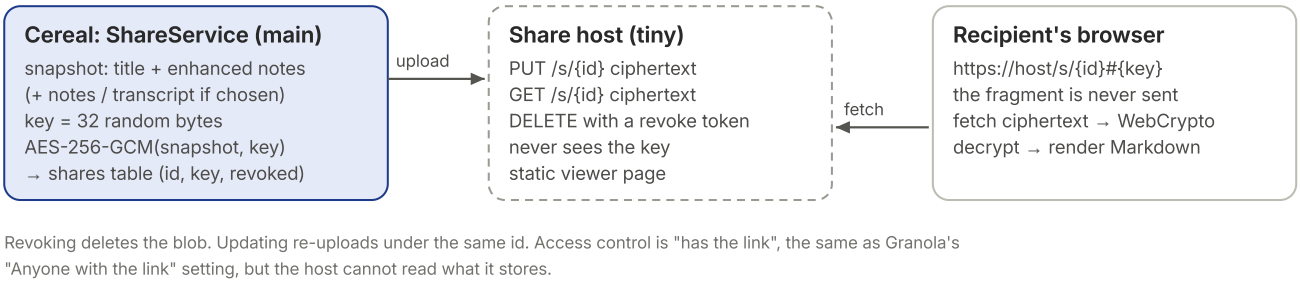


Figure 25.1 A local-first share link. The host can be any object store with a static page (Cloudflare R2 plus a Worker, S3, or a self-hosted box), configured by URL in `.env`.

- **Data.** Migration 6 adds `shares(id, document_id, panel_id, includes, key, revoke_token, url, created_at, updated_at, revoked_at)`.
- **IPC.** `shares:create|update|revoke|list`, plus a `shares:changed` push.
- **UI.** The note's Share menu gains "Create link…": choose what to include, with the transcript off by default, then Copy link, Update and Revoke.
- **Agent.** Optionally a `shareNote` tool with `policy: 'ask'`.
- **Limits.** Snapshots are static (no live updates or comments), and there is no "only my company". Those need identity.

2. Deeper Google connections without a server

- **Incremental scopes on the existing OAuth client.** Keep `calendar.events.readonly` for the basics. When a feature is first enabled, run the same loopback and PKCE flow again with `include_granted_scopes=true` and the extra scope: `gmail.readonly` for email context, `gmail.compose` to create real drafts, `calendar.events` to create events, `drive.readonly` or `documents.readonly` for Docs. Record the granted scopes in settings, and show per-feature connect buttons like Granola's.
- **Agent tools**, mirroring Granola's names:
  - `searchEmailThreads`, `readEmailThread`, `searchGoogleDocs`, `readGoogleDoc`: read-only, so allowed;
  - `createGmailDraft` and `createCalendarEvent`: `policy: 'ask'`;
  - a `requestGoogleAccess` tool that returns a card with a Connect button when a scope is missing, like Granola's `resolveConnectorAccess`.
- **Cheaper polling.** Switch calendar sync to Google's incremental `syncToken`: one cheap request every minute instead of a 15-day window every 5 minutes. Read selected calendars, not just `primary`.

GOTCHAS

- **Testing mode.** A Google OAuth client left in *Testing* publishing status issues refresh tokens that expire after 7 days, and only listed test users can consent. Users must publish their client "In production" for personal use, or re-connect weekly.
- **Restricted scopes.** Gmail scopes are restricted. A personal, unverified client works for its own owner but shows a warning screen.
- **Prompt injection.** Email content is untrusted input to the agent. Keep every write tool behind approval.

3. Integrations through tokens and MCP

- **Slack:** an incoming-webhook URL or a bot token in `.env`, and a `postSlackMessage` tool with `policy: 'ask'`. No OAuth redirect hosting is needed.
- **Notion:** an internal-integration token, and "Save to Notion" creating a page in a chosen database.
- **Generic:** outgoing webhooks on note-enhanced events (Zapier and n8n can receive them).
- **MCP client.** Deferred in the agent plan, but the highest-leverage item. Let the Assistant connect to local or remote MCP servers (stdio or HTTP) configured in a file, expose their tools in the `conversation` and `task` contexts, and default every MCP tool to `ask` unless allowlisted. Granola's chat already supports user-added MCP servers.

4. Sync, if it is ever wanted

1. **Make documents mergeable first.** Move notes and panels to Yjs, as Granola did: TipTap's collaboration extension binds an editor to a Y.Doc, and the state goes in a `ydocs` table. Text-set attribution (`generated_lines`) keeps working, because it matches text, not positions.
2. **Then pick a transport.** A self-hostable relay (a y-websocket server) or a sync service. Keep the WebSocket "hint, then pull" pattern from chapter 19. Encrypt updates end to end if the relay is untrusted.
3. **Leave the rest local.** Transcripts are append-only and can sync as immutable chunks; agent state can stay per-device.

This is a large change. The share links in step 1 cover most of what people want from "send my notes to someone" without it.

5. Smaller wins

Item	Approach	Effort
Updates	Publish to Flathub, or host an OSTree repo (`flatpak build-update-repo` + `flatpak remote-add`); in-app "update available" via the remote.	Small
Echo cancellation	Load PipeWire's `echo-cancel` module and capture its source, or document it.	Small
Diarization of "Them"	Deepgram `diarize=true` on the system stream, with speaker ids in chunk metadata and a rename UI.	Medium
Encrypted database	Swap to `better-sqlite3-multiple-ciphers`, with a random key sealed by async `safeStorage`, mirroring Granola's DEK.	Medium
Semantic search	`sqlite-vec` plus embeddings of panels and transcript turns; a `semanticSearchMeetings` tool.	Medium
Web search	Enable Claude's server-side `web_search` tool in chat (provider-executed, like Granola), or a search API key.	Small
`askUserQuestion`	A structured clarifying-question tool, Granola's shape: 1–3 questions with 2–4 options, answered with a card.	Small

APPENDIX A

Source file map

Every main, preload and shared file, and the renderer by area. Line counts are approximate.

File	Responsibility
src/main	
`index.ts`	Role dispatcher: the app, or the widget host with `--cereal-widget-host`.
`app.ts` (660)	Composition root: services, IPC handlers, windows, tray, timers, lifecycle, attachment protocol.
`ipc.ts`	Typed `handle` with a sender check; `broadcast` to all windows.
`windows.ts`	`loadPage`, `isOwnPage`, `hardenWindow`, widget window options.
`settings.ts` / `settings.pure.ts`	Preferences, `.env` keys (safe reads, 0600), token sealing with async `safeStorage`.
`recorder.ts`	Capture → engine → chunks; interim/final semantics; recording state machine.
`detector.ts`	Meeting start and end from PipeWire polling (3 s polls, 20 s grace).
`calendar.ts`	Google OAuth (loopback + PKCE), event sync and mapping, `current()`.
`audio/pipewire.ts`	`pw-record` capture with RMS levels; device list; meeting-app detection.
`transcription/types.ts`	Engine, stream and segment interfaces.
`transcription/deepgram.ts`	Live WebSocket streaming, keepalive, reconnect with a continuous clock, credential test.
`transcription/whisper.ts`	`whisper-server` lifecycle and serial inference; stream with partials and finals.
`transcription/segmenter.ts`	Energy VAD with Granola's timings; WAV encoding; `PrefixStabilizer`.
`transcription/models.ts`	ggml model list and downloads.
`transcription/dictation.ts`	Chat-bar dictation through Deepgram prerecorded or a warm whisper-server.
`ai/service.ts`	Enhance, rewrite, save edits, condense, auto-title, citation extraction.
`ai/variables.ts` / `ai/prompts.ts`	Prompt variables; Cereal's own prompts (enhance, condense, Transcript Summary, title).
`ai/transcriptSummary.ts`	On-demand "where did this line come from", streamed.
`llm/provider.ts`	Provider interface, neutral transcript types, tool specs, files access.
`llm/index.ts`	Config resolution, provider factory, model picker list, error descriptions.
`llm/{anthropic,openai,gemini,ollama}.ts`	Wire translation, streaming, tool calling, replay, Files APIs.
`llm/files.ts`	`uploadOnce`, attachment text forms.
`attachments/service.ts` / `extract.ts`	Validation, storage, text extraction, remote cleanup, purge.
`agent/protocol.ts`	The runtime ↔ backend contract.
`agent/runtime.ts`	The model/tool loop, event batching, compaction.
`agent/backend.ts`	Runs, tool execution and approvals, event folding, finishing.
`agent/host.ts` / `worker-host.ts` / `worker/entry.ts`	In-process host; utility-process host; the worker's proxy and port.
`agent/chat.ts`	Ask Cereal: send, retry, rewind, prompts, titles, client-applied edits.
`agent/assistant.ts`	Conversation, tasks, inbox drain, routines tick, Home, notifications.

File	Responsibility
`agent/schedule.ts`	`at` / `rate` / `cron` parsing and next-fire times in timezones.
`agent/files.ts`	Sandboxed agent folder: memory, skills, files.
`agent/prompts.ts` / `tool.ts`	Agent prompts; `defineTool`, specs, policies, argument parsing.
`agent/tools/*.ts`	Meetings and calendar, notes, drafts, files, assistant orchestration.
`db/index.ts` / `migrations.ts`	Open with pragmas; ordered migrations by `user_version`.
`db/store.ts` / `agent-store.ts`	All SQL: notes, chat, search · agent runs, events, threads, inbox, routines.
`widget/controller.ts` / `host.ts` / `protocol.ts` / `placement.ts`	In-process or XWayland-hosted widget, NDJSON protocol, placement maths.
src/preload	
`index.ts`	`window.cereal`: `invoke`, `on` and `platform`, behind channel allowlists.
src/shared	
`ipc.ts` / `types.ts`	The IPC contract; domain types.
`agent.ts`	Agent types, events, the outputs reducer `applyAgentEvent`.
`transcript.ts`	Chunk merge, grouping, formatting, find, heading budget.
`markdown.ts`	ProseMirror JSON ↔ Markdown and plain text, line parsing.
`attribution.ts` / `generatedLines.ts`	User vs AI line detection; decorations for grey lines.
`transcriptSummary.ts`	Chunk selection, reply parsing, `matchQuote`.
`models.ts` / `chatbar.ts` / `attachments.ts`	Model picker data; chat-bar placeholders, triggers and recipe ranking; attachment rules.
`defaults.ts` / `barLevels.ts`	Default settings, built-in templates and recipes; the level noise gate.
src/renderer/src	
`App.tsx`, `main.tsx`, `store.ts`, `lib/*`	Boot, routes, global stores, IPC helpers, router, sounds, formatting, throttle, note edits.
`components/shell/*`	Sidebar, toolbar, layout store.
`views/*`	Home, Note (and `note/*`), Folder, Chat, Assistant, Recipes, Settings.
`components/chat/*`, `FloatingChat.tsx`, `Chat.tsx`, `AgentOutputs.tsx`	Composer, menus, chips, model picker, dictation; chat surfaces; message rendering.
`components/oats/*`	Design-system primitives, motion, WebGL dancing bars.
`widget.tsx`	The floating pill and meeting prompt.

APPENDIX B

Glossary

Term	Meaning
Chunk	One piece of transcript from one source, with ISO start and end and `is_final`. Interims keep their id until finalised.
Panel	One "Enhanced notes" tab: Markdown plus TipTap JSON plus `generated_lines`.
Template	Headings and descriptions that shape enhanced notes.
Recipe	A saved chat prompt run with `/slug`.
Generated lines	The set of line texts the AI wrote, shown grey; edited lines stop matching.
Run	One execution of the agent loop for one input.
Epoch	A segment of a thread's append-only model transcript; a new one starts on compaction or edit.
Provider replay	The provider-native assistant content (thinking, encrypted reasoning, signatures), replayed verbatim.
Context (tool)	`note-chat`, `global-chat`, `conversation` or `task`: decides which tools a run gets.
Inbox / drain	The queue of inputs for Assistant threads, and the step that turns them into runs.
Routine	A scheduled message to the Assistant (`at`, `rate`, `cron` + timezone).
Nub	Granola's name for the floating recording widget.
Sink monitor	PipeWire's tap on an output device, which is how Cereal hears "Them".
Keto	Granola's cloud Assistant runtime (worker + gateway + socket).
WorkOS	The identity provider behind Granola's sign-in (Google, Microsoft, SSO).
Envelope encryption	A random data key encrypts the data; the OS keyring encrypts the data key (Granola's `storage.dek`).
REA	The reverse-engineering toolkit used for the static analysis of Granola.

APPENDIX C

Evidence and further reading

In the repository

- `README.md`: features, requirements, install, develop, architecture summary.
- `docs/granola-findings.md`: the main behaviour spec, with the reconstruction checklist mapping each Granola behaviour to Cereal code and its test.
- `docs/granola-agents.md`: all six Granola agent surfaces.
- `docs/granola-ui.md`: UI, UX and motion values.
- `docs/granola-chat-bar.md`: the chat bar, from 7.668.0.

Evidence files (never commit)

File in `~/re/granola/evidence/`	Covers
`granola-7.663.2-evidence.json`	Fuses, native module, application graph (first pass).
`granola-7.663.2-ui-evidence.json`	UI chunks (`ev_1e8338e0…`).
`granola-7.663.2-agents-evidence.json`	Agent chunks (`ev_82a83c73…`).
`granola-7.668.0-chatbar-evidence.json`	Chat bar and the version comparison (`ev_b74c0660…`, `ev_c03e0c90…`).
`granola-7.668.0-platform-evidence.json`	This guide's pass: auth, sync, sharing, Google, integrations, updates. Parent `ev_c3cea9ae6db7bab75de44c01154d23f288a5cbad0ad8fb16b876d7f921835835`.

This pass's source regions (7.668.0, read as inert text)

Chunk	Facts taken from it
`dist-app/assets/types-64ZdlfKL.js`	The endpoint URL map and hosts; visibility enums; WebSocket URLs.
`dist-app/assets/api-3_SNGkOB.js`	Request wrappers, queues, headers; auth, calendar, Google, integrations and workspace schemas.
`dist-app/assets/getAPIEndpointInWindow-VAm56kka.js`	The local SQLite schema (Drizzle migrations).
`dist-app/assets/TranscribeIcon-D43rFzdJ.js`	Full, partial and entity sync; the Yjs migration and conflict handling.
`dist-app/assets/primary-CUZSZ_4J.js`	The WebSocket client and message types; the share dialog; integration connect and polling; calendar refresh triggers.
`dist-app/assets/index-zj2gijRl.js`	The web-shell bridge stub and session storage.
`dist-app/assets/frecency-DvNc69Wk.js`, `constants-BnNe1_lR.js`	Note link formats; roles.
`dist-electron/main/index.js`	Deep-link scheme and router; WorkOS tokens; header injection; DEK and encrypted stores; the cache state; auto-update.

Static analysis observes what the client contains, not what runs or what the server does. Server-side behaviour in Part II is inferred and marked `I`. Server prompt text was never seen.